

BAB IX

CONSTRAINTS DAN VIEWS

9.1 Tujuan Praktikum

Praktikum ini bertujuan agar mahasiswa memahami :

1. Mahasiswa diharapkan mampu mengelola `CONSTRAINT` menggunakan perintah seperti `ENABLE`, `DISABLE`, `DROP CONSTRAINT`, `DROP COLUMN`, `CASCADE`, serta `CASCADE CONSTRAINT` dalam pengolahan struktur tabel.
2. Mahasiswa memahami cara membangun relasi antar tabel dengan memanfaatkan `REFERENCES` melalui penggunaan `FOREIGN KEY`.
3. Mahasiswa mampu memahami teknik pengolahan dan pembatasan hasil *query*, seperti penggunaan *subquery*, *inline view*, `ROWNUM`, *TOP-N analysis*, serta opsi `WITH CHECK OPTION` dan `WITH READ ONLY`.
4. Mahasiswa diharapkan memahami penerapan berbagai jenis *constraint* seperti `PRIMARY KEY`, `FOREIGN KEY`, `NOT NULL`, `UNIQUE`, dan `CHECK` pada level kolom maupun tabel untuk menjaga integritas data.
5. Mahasiswa memahami pembuatan serta pembaruan `VIEW` menggunakan perintah `CREATE VIEW`, `CREATE OR REPLACE VIEW`, `FORCE`, `NOFORCE`, serta penghapusan `VIEW` dengan `DROP VIEW`.

9.2 Dasar Teori

`Constraint` merupakan sekumpulan regulasi dalam basis data yang berfungsi menjaga integritas serta validitas informasi yang diinput. Komponen-komponen utamanya meliputi `Primary Key`, `Foreign Key`, `Not Null`, `Unique`, dan `Check`. Bergantung pada kompleksitas relasi data yang diinginkan, aturan-aturan ini dapat diimplementasikan pada tingkatan kolom maupun tabel (Oracle, 2020).

`Foreign Key` berperan sebagai jembatan yang menghubungkan antar tabel melalui kolom yang saling berkaitan. Dengan memanfaatkan klausa `REFERENCES` untuk menentukan acuan pada tabel induk, aturan ini menjamin terciptanya integritas referensial. Hal ini memastikan bahwa setiap data dalam tabel anak selalu

sinkron dengan tabel utama, yang merupakan aspek krusial dalam menjaga konsistensi sistem basis data relasional (Oracle, 2020).

`VIEW` adalah tabel virtual yang dihasilkan dari sebuah *query* terhadap satu atau lebih tabel dasar. Perintah `CREATE VIEW` digunakan untuk membuat `VIEW` baru, sedangkan `CREATE OR REPLACE VIEW` digunakan untuk memperbarui `VIEW` yang sudah ada sebelumnya. Opsi `FORCE` memungkinkan pembuatan `VIEW` meskipun tabel sumber belum tersedia, sementara `NOFORCE` hanya akan membuat `VIEW` jika tabel dasar sudah ada (Oracle, 2020).

Pengelolaan `CONSTRAINT` dapat dilakukan menggunakan berbagai perintah seperti `ENABLE`, `DISABLE`, `DROP CONSTRAINT`, `DROP COLUMN`, `CASCADE`, dan `CASCADE CONSTRAINT`. `ENABLE` digunakan untuk mengaktifkan kembali `CONSTRAINT`, sedangkan `DISABLE` digunakan untuk menonaktifkannya sementara tanpa menghapusnya. `DROP CONSTRAINT` berfungsi menghapus aturan tertentu, dan `DROP COLUMN` digunakan untuk menghapus kolom dari tabel. Sementara itu, `CASCADE` dan `CASCADE CONSTRAINT` digunakan untuk menangani penghapusan yang melibatkan objek lain yang memiliki keterkaitan (Oracle, 2020).

Dalam pengelolaan `VIEW`, terdapat beberapa opsi tambahan seperti `WITH CHECK OPTION` dan `WITH READ ONLY`. `WITH CHECK OPTION` memastikan bahwa perubahan data melalui `VIEW` tetap sesuai dengan kondisi *query* pembentuknya. `WITH READ ONLY` digunakan untuk membatasi `VIEW` agar hanya dapat dibaca tanpa bisa dimodifikasi. Selain itu, `DROP VIEW` digunakan untuk menghapus `VIEW` tanpa memengaruhi tabel dasar. Teknik seperti *subquery*, *inline view*, `ROWNUM`, dan *TOP-N analysis* juga sering digunakan untuk menampilkan data secara lebih spesifik (Oracle, 2020).

`ON DELETE CASCADE` dan `ON DELETE SET NULL` merupakan aturan yang digunakan untuk mengatur dampak penghapusan data pada tabel induk terhadap tabel anak. `ON DELETE CASCADE` akan secara otomatis menghapus data yang terkait, sedangkan `ON DELETE SET NULL` akan mengubah nilai `FOREIGN KEY` pada tabel anak menjadi `NULL`. Kedua mekanisme ini penting untuk menjaga konsistensi data dan menghindari adanya data yang tidak memiliki referensi (Oracle, 2020).

Tabel berikut menyajikan penjelasan mengenai berbagai sintaks yang digunakan dalam pembuatan *view* pada *database*. Setiap sintaks dijelaskan secara ringkas untuk membantu memahami fungsi dan penggunaannya dalam perintah SQL.

Tabel 9.1 Tabel deskripsi klausa pembuatan *VIEW*

Sintaks	Penjelasan
OR REPLACE	Membuat ulang <i>VIEW</i> jika <i>VIEW</i> tersebut sudah ada sebelumnya.
FORCE	Membuat <i>VIEW</i> tanpa mempedulikan apakah tabel dasar sudah ada atau belum.
NOFORCE	Membuat <i>VIEW</i> hanya jika tabel dasar sudah ada.
<i>view_name</i>	Menentukan nama untuk <i>VIEW</i> yang akan dibuat.
<i>alias</i>	Menentukan nama alias untuk setiap ekspresi/kolom yang dipilih oleh <i>query</i> di dalam <i>VIEW</i> .
<i>subquery</i>	Merupakan pernyataan <i>SELECT</i> yang lengkap. Anda dapat menggunakan alias untuk kolom pada daftar <i>SELECT</i> . <i>Subquery</i> ini dapat berisi sintaks <i>SELECT</i> yang kompleks.
WITH CHECK OPTION	Memastikan bahwa baris data tetap dapat diakses oleh <i>VIEW</i> setelah dilakukan operasi <i>INSERT</i> atau <i>UPDATE</i> . Ini mencegah perubahan data yang akan membuat baris tersebut hilang dari kriteria <i>VIEW</i> .
CONSTRAINT	Nama yang diberikan/ditetapkan untuk batasan <i>CHECK OPTION</i> .
WITH READ ONLY	Memastikan bahwa tidak ada operasi <i>DML</i> yang dapat dilakukan pada <i>VIEW</i> ini.

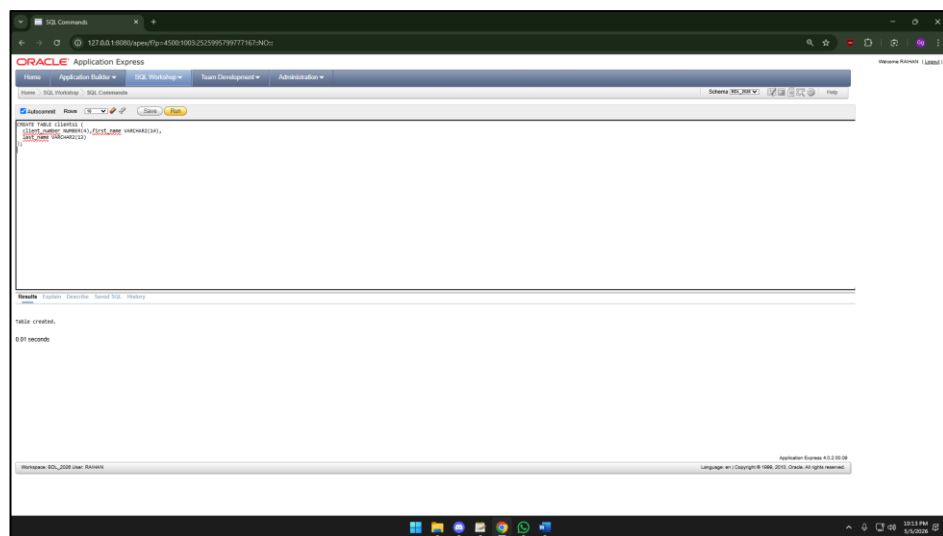
9.3 Hasil dan Pembahasan

Adapun beberapa percobaan *query* dari modul *section* 14 dan 15 yang sudah dipelajari yaitu sebagai berikut:

1. *Query*:

```
CREATE TABLE clients1
(client_number NUMBER(4),
first_name VARCHAR2(14),
last_name VARCHAR2(13));
```

Hasil:



Gambar 9.1 Tampilan *create* tabel *clients1*

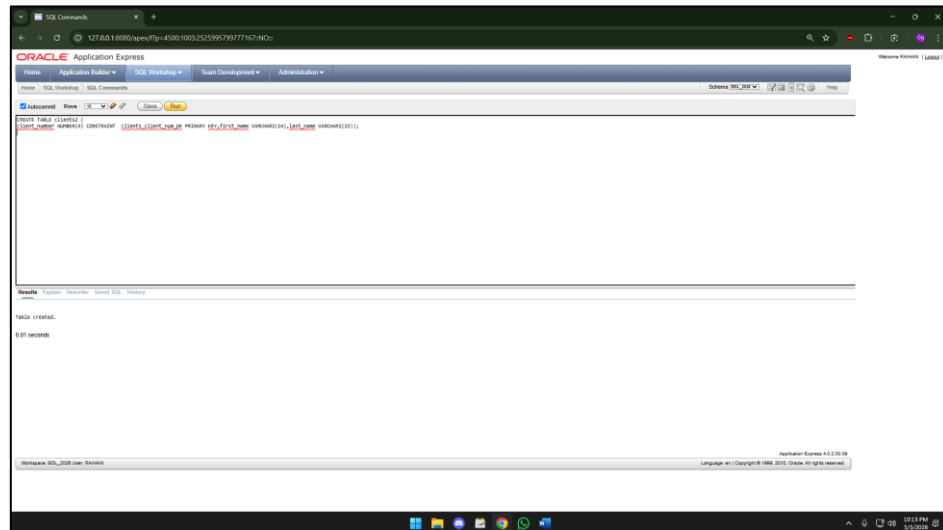
Penjelasan:

Perintah tersebut digunakan untuk membuat sebuah tabel baru dengan nama *clients* yang terdiri dari tiga atribut, yaitu *client_number*, *first_name*, dan *last_name*. Pada prosesnya, setiap kolom ditentukan tipe datanya, seperti *NUMBER(4)* untuk menyimpan nomor klien dan *VARCHAR2* untuk menyimpan nama depan serta nama belakang. Setelah dijalankan, tabel *clients* akan terbentuk dan dapat digunakan sebagai tempat penyimpanan data klien.

2. Query:

```
CREATE TABLE clients2  
(client_number NUMBER(4)  
CONSTRAINT clients_client_num_pk PRIMARY KEY,  
first_name VARCHAR2(14), last_name VARCHAR2(13));
```

Hasil:



Gambar 9.2 Tampilan *create* tabel *clients2* dengan PRIMARY KEY pada kolom *client_number*

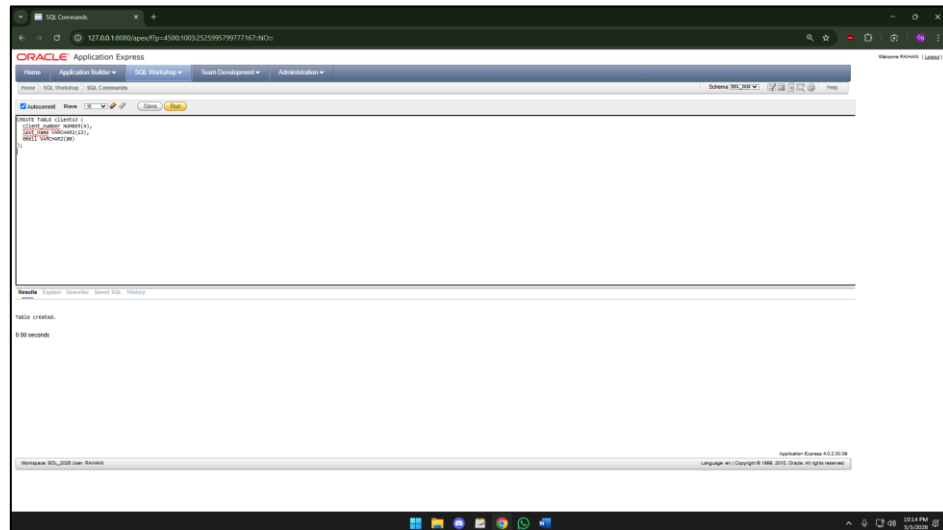
Penjelasan:

Perintah tersebut berfungsi untuk membuat tabel baru dengan nama *clients2* yang memiliki tiga kolom, yaitu *client_number*, *first_name*, dan *last_name*. Dalam definisinya, kolom *client_number* ditetapkan sebagai PRIMARY KEY melalui CONSTRAINT bernama *clients_client_num_pk*, sehingga nilainya harus unik dan tidak boleh bernilai NULL. Dengan demikian, tabel *clients2* berhasil dibuat lengkap dengan aturan kunci utama pada kolom *client_number*.

3. Query:

```
CREATE TABLE clients3  
(client_number NUMBER(4),  
last_name VARCHAR2(13),  
email VARCHAR2(80));
```

Hasil:



Gambar 9.3 Tampilan *create* tabel *clients3* dengan kolom *client_number*, *last_name*, dan *email*

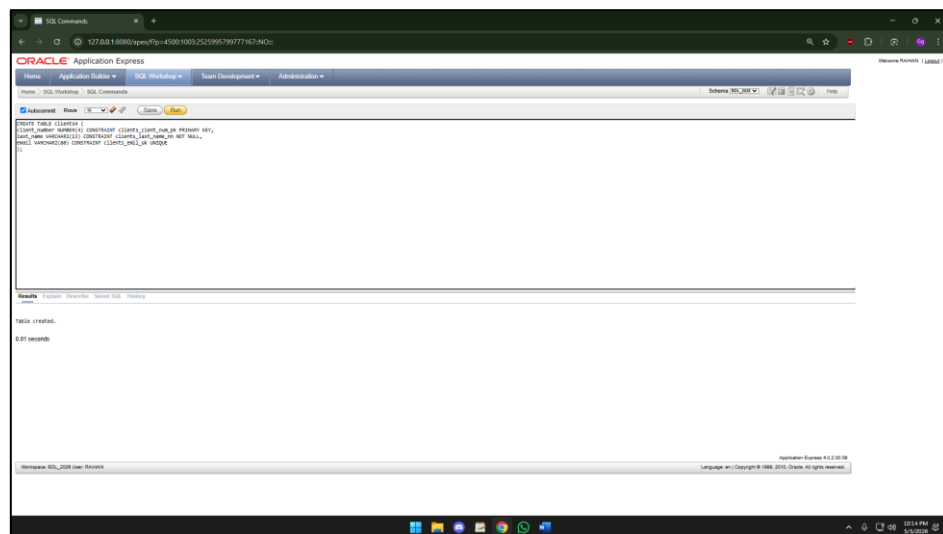
Penjelasan:

Perintah tersebut digunakan untuk membentuk tabel baru dengan nama *clients3* yang terdiri dari tiga kolom, yaitu *client_number*, *last_name*, dan *email*. Pada proses pembuatannya, setiap kolom ditentukan tipe datanya, seperti *NUMBER(4)* untuk menyimpan nomor klien serta *VARCHAR2* untuk menyimpan nama belakang dan alamat email. Setelah *query* dijalankan, tabel *clients3* berhasil dibuat dan dapat digunakan untuk menyimpan data klien beserta informasi emailnya.

4. Query:

```
CREATE TABLE clients4 (client_number NUMBER(4)
CONSTRAINT clients_cient_num_pk PRIMARY KEY,
last_name VARCHAR2(13)
CONSTRAINT clients_last_name_nn NOT NULL,
email VARCHAR2(80)
CONSTRAINT clients_emil_uk UNIQUE);
```

Hasil:



Gambar 9.4 Tampilan *create* tabel *clients4* dengan CONSTRAINT PRIMARY KEY, NOT NULL, dan UNIQUE

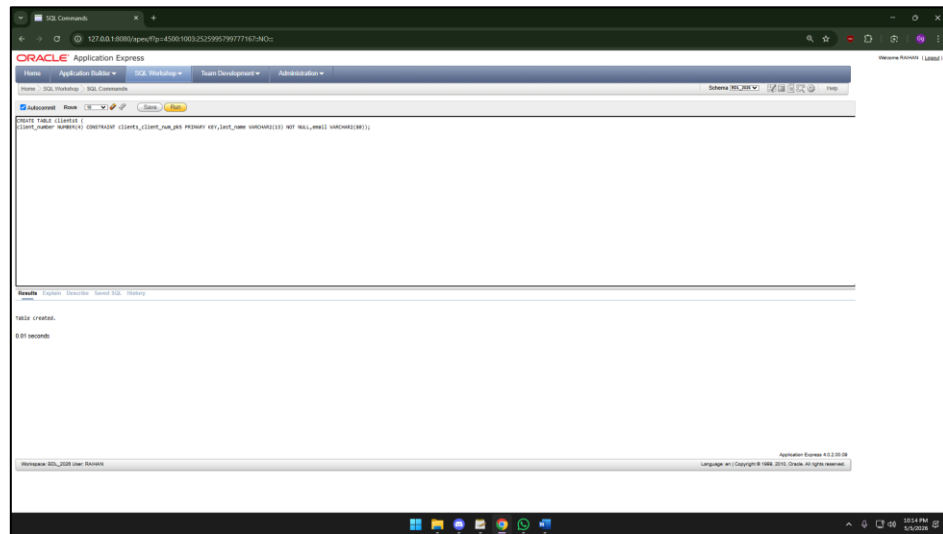
Penjelasan:

Perintah tersebut digunakan untuk membuat tabel baru bernama *clients4* yang memiliki tiga kolom, yaitu *client_number*, *last_name*, dan *email*. Dalam definisinya, kolom *client_number* dijadikan PRIMARY KEY sehingga nilainya harus unik, kemudian *last_name* diberi aturan NOT NULL agar wajib diisi, serta *email* ditetapkan sebagai UNIQUE sehingga tidak boleh ada data yang sama. Dengan demikian, tabel *clients4* berhasil dibuat dengan aturan kunci utama, kewajiban pengisian pada nama belakang, dan pembatasan keunikan pada email setiap klien.

5. Query:

```
CREATE TABLE clients5  
(client_number NUMBER(4)  
CONSTRAINT clients_client_num_pk PRIMARY KEY,  
last_name VARCHAR2(13) NOT NULL,  
email VARCHAR2(80));
```

Hasil:



Gambar 9.5 Tampilan *create* tabel `clients5` dengan PRIMARY KEY dan NOT NULL

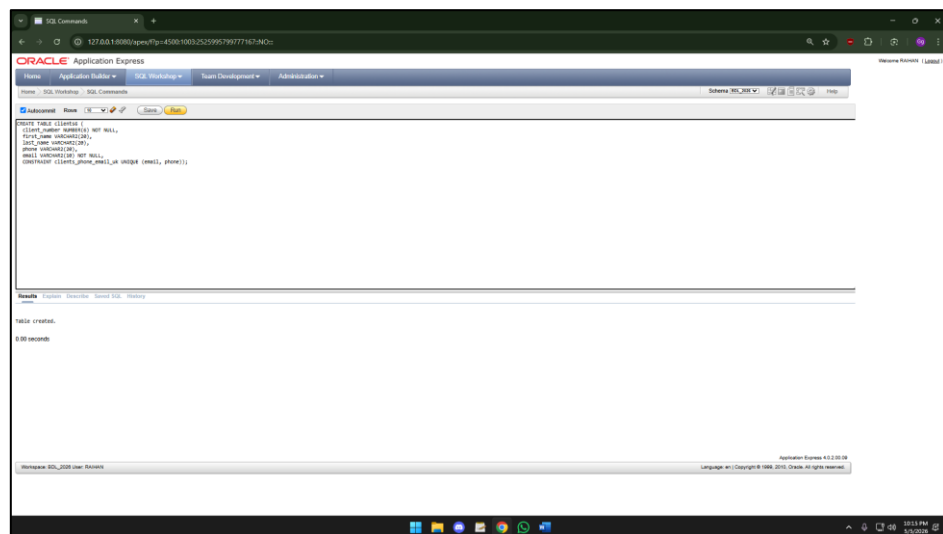
Penjelasan:

Perintah tersebut berfungsi untuk membuat tabel baru dengan nama `clients5`. Pada definisinya, kolom `client_number` ditetapkan sebagai PRIMARY KEY sehingga nilainya harus unik, sementara `last_name` diberi aturan NOT NULL yang berarti wajib diisi. Dengan demikian, tabel `clients5` berhasil dibuat dengan ketentuan kunci utama pada `client_number` serta kewajiban pengisian pada kolom `last_name`.

6. Query:

```
CREATE TABLE clients6
( client_number NUMBER(6) NOT NULL,
  first_name VARCHAR2(20), last_name VARCHAR2(20),
  phone VARCHAR2(20), email VARCHAR2(10) NOT NULL,
  CONSTRAINT clients_phone_email_uk
  UNIQUE (email,phone));
```

Hasil:



Gambar 9.6 Tampilan *create* tabel *clients6* dengan CONSTRAINT UNIQUE

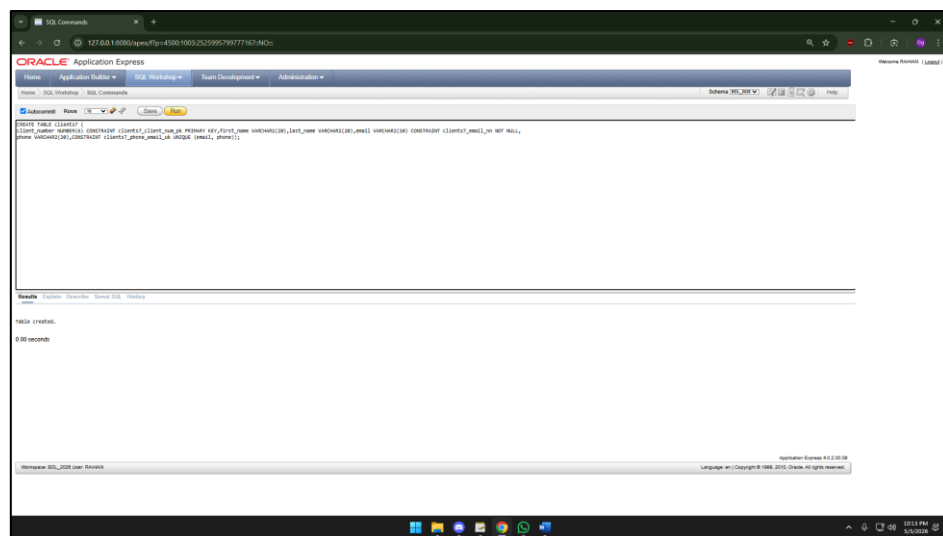
Penjelasan:

Perintah `CREATE TABLE` digunakan untuk membuat tabel *clients6* dengan kolom *client_number*, *first_name*, *last_name*, *phone*, dan *email*. Pada definisinya, *client_number* dan *email* ditetapkan sebagai `NOT NULL` sehingga wajib diisi, serta diberikan `CONSTRAINT UNIQUE` pada kombinasi *email* dan *phone*. Hasilnya, tabel *clients6* terbentuk dengan aturan pengisian wajib dan pembatasan agar kombinasi *email* dan *phonenya* tidak sama.

7. Query:

```
CREATE TABLE clients7 (  
  client_number NUMBER(6) CONSTRAINT  
  clients7_client_num_pk PRIMARY KEY, first_name  
  VARCHAR2(20), last_name VARCHAR2(20), email  
  VARCHAR2(10) CONSTRAINT clients7_email_nn NOT NULL,  
  phone VARCHAR2(20), CONSTRAINT  
  clients7_phone_email_uk UNIQUE (email, phone));
```

Hasil:



Gambar 9.7 Tampilan *create* tabel `clients7`

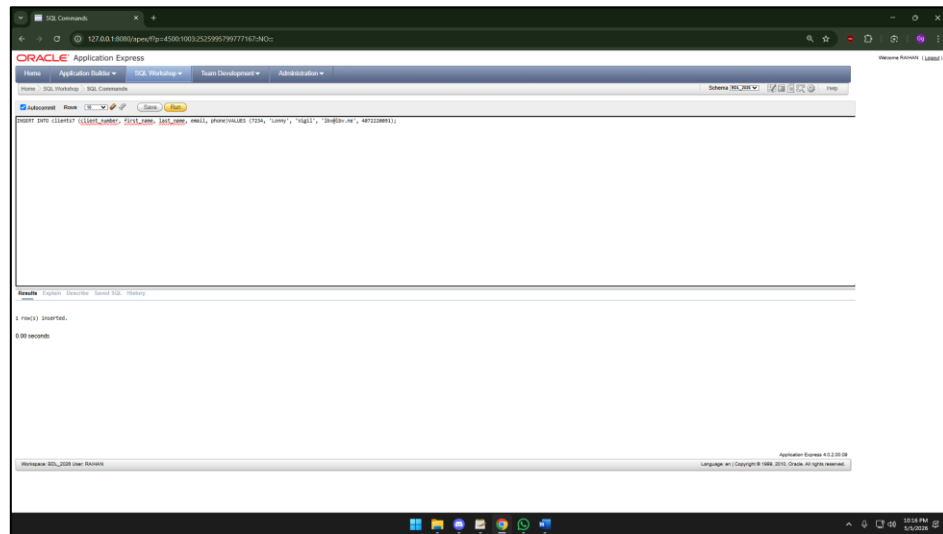
Penjelasan:

Perintah tersebut digunakan untuk membuat tabel `clients7` yang terdiri dari kolom `client_number`, `first_name`, `last_name`, `email`, dan `phone` dengan berbagai aturan data. Kolom `client_number` ditetapkan sebagai `PRIMARY KEY` untuk identitas unik, kolom `email` diberikan batasan `NOT NULL` agar wajib diisi, serta terdapat batasan `UNIQUE` gabungan untuk memastikan kombinasi email dan nomor telepon tidak ada yang kembar. Setelah dijalankan, tabel ini akan terbentuk dengan struktur validasi yang memastikan integritas data klien yang disimpan di dalamnya.

8. Query:

```
INSERT INTO clients7  
(client_number, first_name, last_name,  
phone, email)  
VALUES (7234, 'Lonny', 'Vigil',  
4072220091, 'lbv@lbv.net');
```

Hasil:



Gambar 9.8 Tampilan penambahan data “Lonny Vigil” ke dalam tabel `clients7`

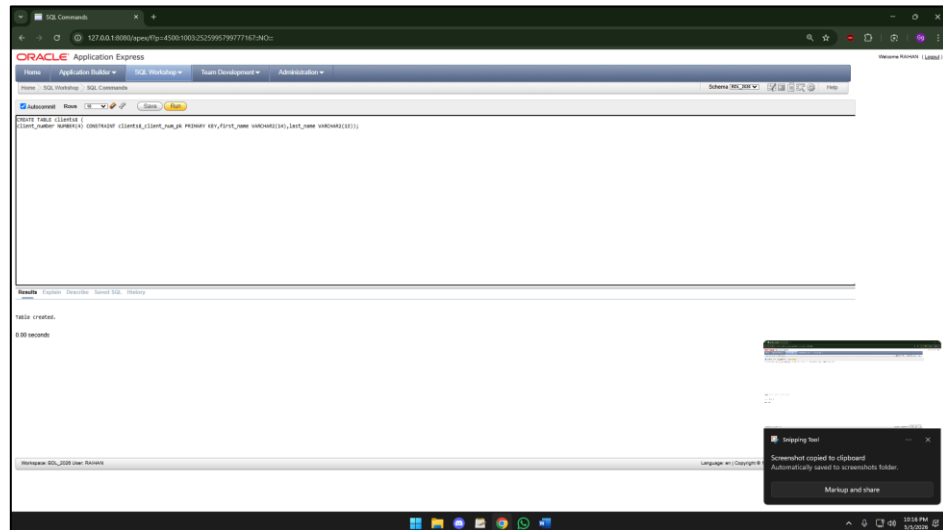
Penjelasan:

Perintah `INSERT INTO` digunakan untuk menambahkan satu data baru ke tabel `clients7`, yaitu dengan nilai `client_number` “7234”, `first_name` “Lonny”, `last_name` “Vigil”, `phone` “4072220091”, dan `email` “lbv@lbv.net”. Data dimasukkan sesuai urutan kolom yang telah ditentukan. Hasilnya, data tersebut berhasil ditambahkan karena kombinasi `email` dan `phone` tidak duplikat dengan data yang sudah ada.

9. Query:

```
CREATE TABLE clients8  
(client_number NUMBER(4)  
CONSTRAINT clients_client_num_pk PRIMARY KEY,  
first_name VARCHAR2(14), last_name VARCHAR2(13));
```

Hasil:



Gambar 9.9 Tampilan *create* tabel `clients8` dengan PRIMARY KEY pada kolom `client_number`

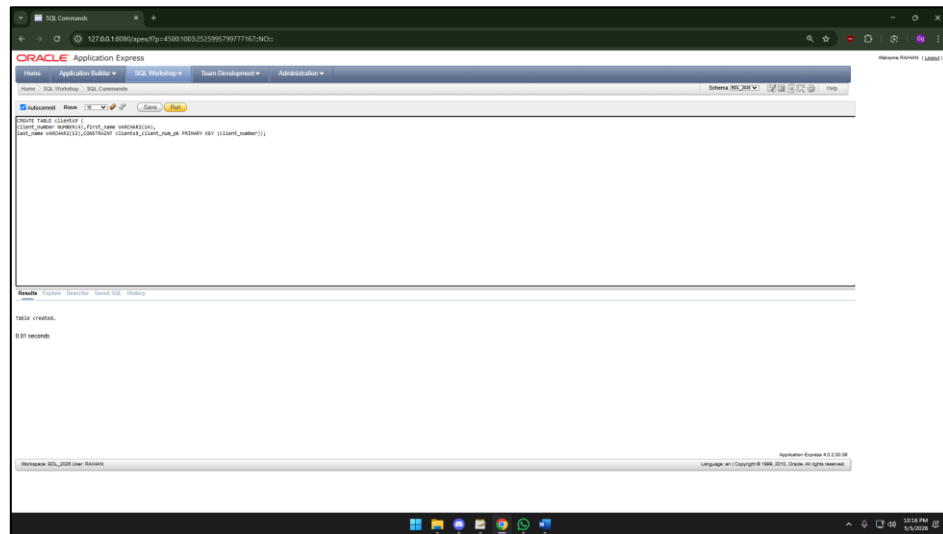
Penjelasan:

Perintah `CREATE TABLE` digunakan untuk membuat tabel `clients8` dengan kolom `client_number`, `first_name`, dan `last_name`. Pada definisinya, `client_number` ditetapkan sebagai PRIMARY KEY melalui `CONSTRAINT clients_client_num_pk`, sehingga nilainya harus unik dan tidak boleh NULL. Hasilnya, tabel `clients8` berhasil dibuat dengan aturan kunci utama pada kolom tersebut.

10. Query:

```
CREATE TABLE clients9 (  
  client_number NUMBER(4), first_name VARCHAR2(14),  
  last_name VARCHAR2(13), CONSTRAINT  
  clients9_client_num_pk PRIMARY KEY  
  (client_number));
```

Hasil:



Gambar 9.10 Tampilan *create* tabel *clients9* dengan CONSTRAINT PRIMARY KEY pada kolom *client_number*

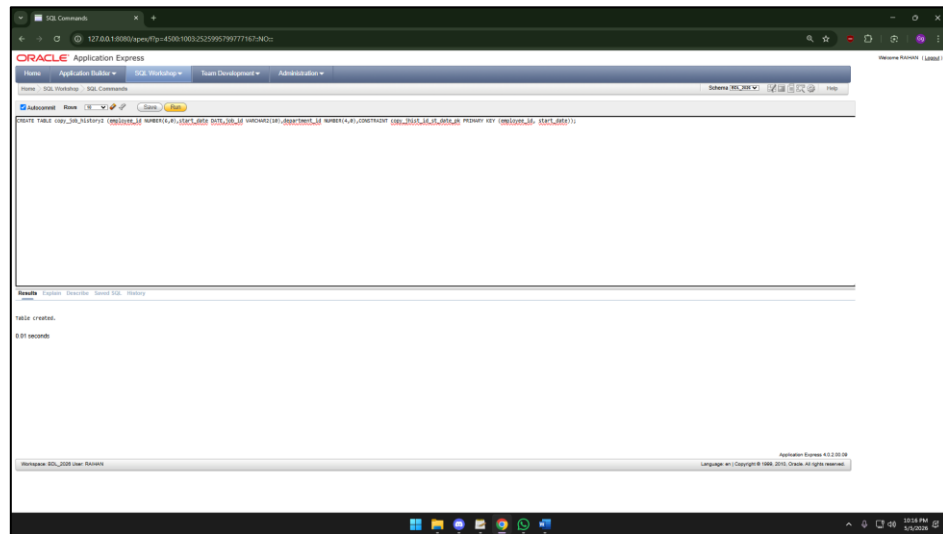
Penjelasan:

Perintah `CREATE TABLE` digunakan untuk membuat tabel *clients9*. Pada definisinya, *client_number* dijadikan PRIMARY KEY melalui CONSTRAINT *clients9_client_num_pk* yang ditulis pada level tabel. Hasilnya, tabel *clients9* berhasil dibuat dengan ketentuan kunci utama pada kolom tersebut.

11. Query:

```
CREATE TABLE copy_job_history2
(employee_id NUMBER(6,0), start_date DATE,
job_id VARCHAR2(10), department_id NUMBER(4,0),
CONSTRAINT copy_jhist_id_st_date_pk
PRIMARY KEY(employee_id, start_date));
```

Hasil:



Gambar 9.11 Tampilan *create* tabel `copy_job_history2` dengan PRIMARY KEY gabungan pada kolom `employee_id` dan `start_date`

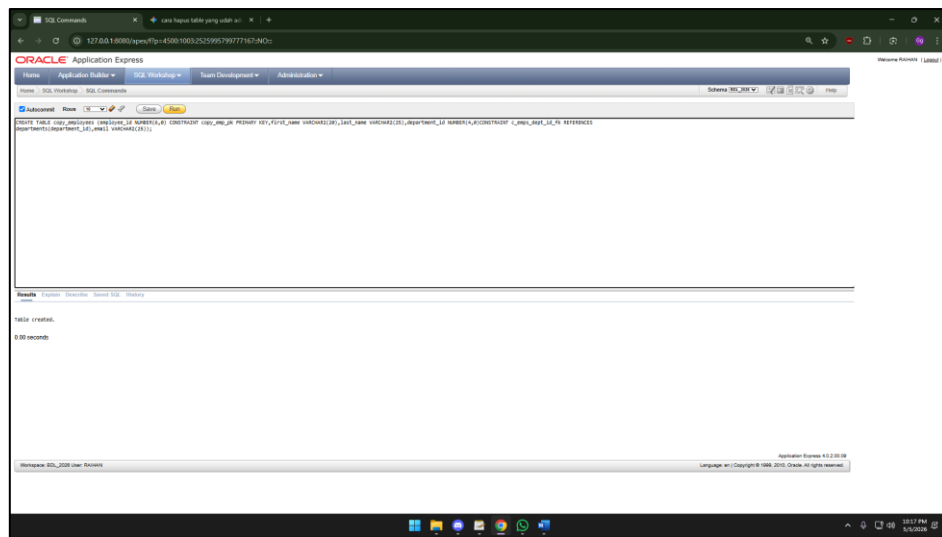
Penjelasan:

Perintah `CREATE TABLE` digunakan untuk membuat tabel `copy_job_history2`. Pada definisinya, kombinasi `employee_id` dan `start_date` ditetapkan sebagai PRIMARY KEY melalui CONSTRAINT `copy_jhist_id_st_date_pk`. Hasilnya, tabel berhasil dibuat dengan kunci utama gabungan sehingga tidak ada data dengan kombinasi tersebut yang sama.

12. Query:

```
CREATE TABLE copy_employees
(employee_id NUMBER(6,0)
CONSTRAINT copy_emp_pk PRIMARY KEY,
first_name VARCHAR2(20), last_name VARCHAR2(25),
department_id NUMBER(4,0)
CONSTRAINT c_emps_dept_id_fk REFERENCES
departments(department_id), email VARCHAR2(25));
```

Hasil:



Gambar 9.12 Tampilan *create* tabel *copy_employees* dengan
PRIMARY KEY dan FOREIGN KEY

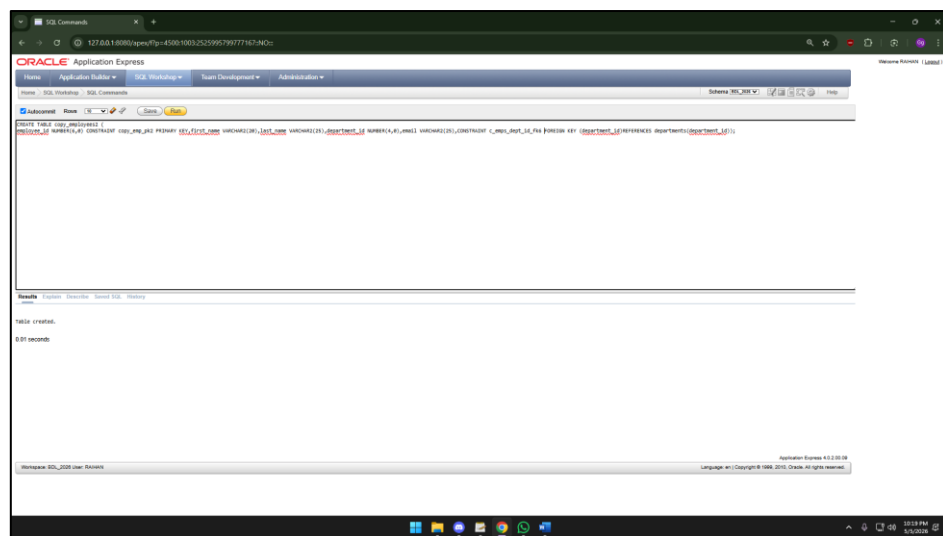
Penjelasan:

Perintah `CREATE TABLE` digunakan untuk membuat tabel *copy_employees*. Pada definisinya, *employee_id* ditetapkan sebagai `PRIMARY KEY`, sedangkan *department_id* dijadikan `FOREIGN KEY` yang mereferensikan kolom *department_id* pada tabel *departments*. Hasilnya, tabel berhasil dibuat dengan kunci utama serta relasi antar tabel melalui kolom tersebut.

13. Query:

```
CREATE TABLE copy_employees2 (employee_id  
NUMBER(6,0) CONSTRAINT copy_emp_pk PRIMARY KEY,  
first_name VARCHAR2(20), last_name VARCHAR2(25),  
department_id NUMBER(4,0), email VARCHAR2(25),  
CONSTRAINT c_emps_dept_id_fk FOREIGN KEY  
(department_id) REFERENCES  
departments(department_id));
```

Hasil:



Gambar 9.13 Tampilan *create* tabel *copy_employees* dengan
PRIMARY KEY dan FOREIGN KEY

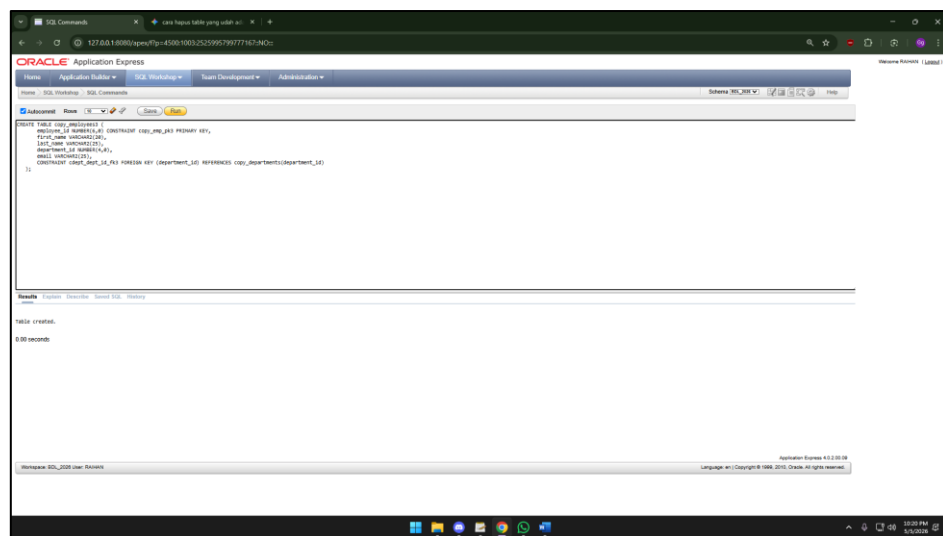
Penjelasan:

Perintah `CREATE TABLE` digunakan untuk membuat tabel *copy_employees2*. Pada definisinya, *employee_id* ditetapkan sebagai `PRIMARY KEY` sehingga setiap data memiliki identitas unik. Selain itu, *department_id* dijadikan `FOREIGN KEY` yang mengacu pada kolom *department_id* di tabel *departments* melalui `CONSTRAINT c_emps_dept_id_fk`. Hasilnya, tabel *copy_employees2* berhasil dibuat dengan aturan kunci utama serta hubungan antar tabel yang menjaga konsistensi data.

14. Query:

```
CREATE TABLE copy_employees3 (  
  employee_id NUMBER(6,0) CONSTRAINT copy_emp_pk3  
  PRIMARY KEY, first_name VARCHAR2(20), last_name  
  VARCHAR2(25), department_id NUMBER(4,0), email  
  VARCHAR2(25), CONSTRAINT cdept_dept_id_fk3 FOREIGN  
  KEY (department_id) REFERENCES  
  copy_departments(department_id));
```

Hasil:



Gambar 8.14 Tampilan *create* tabel *copy_employees* dengan
FOREIGN KEY

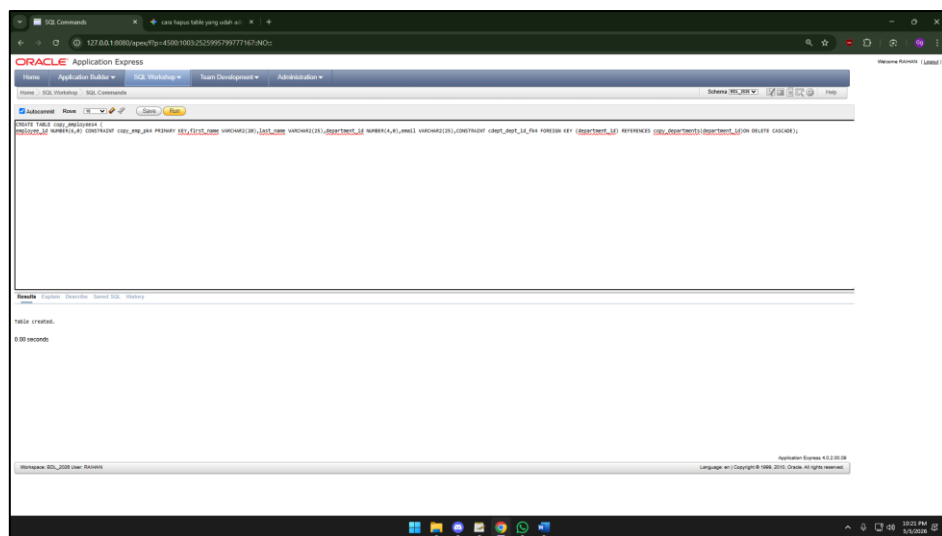
Penjelasan:

Perintah `CREATE TABLE` digunakan untuk membuat tabel `copy_employees3`. Dalam definisinya, `employee_id` ditetapkan sebagai `PRIMARY KEY` sehingga setiap data bersifat unik. Selain itu, `department_id` dijadikan `FOREIGN KEY` yang mengacu pada kolom `department_id` di tabel `copy_departments` melalui `CONSTRAINT cdept_dept_id_fk`. Hasilnya, tabel `copy_employees3` berhasil dibuat dengan kunci utama serta relasi ke tabel `copy_departments`.

15. Query:

```
CREATE TABLE copy_employees4 (  
  employee_id NUMBER(6,0) CONSTRAINT copy_emp_pk4  
  PRIMARY KEY, first_name VARCHAR2(20), last_name  
  VARCHAR2(25), department_id NUMBER(4,0), email  
  VARCHAR2(25), CONSTRAINT cdept_dept_id_fk4 FOREIGN  
  KEY (department_id) REFERENCES  
  copy_departments(department_id) ON DELETE CASCADE);
```

Hasil:



Gambar 9.15 Tampilan *create* tabel `copy_employees` dengan ON
DELETE CASCADE

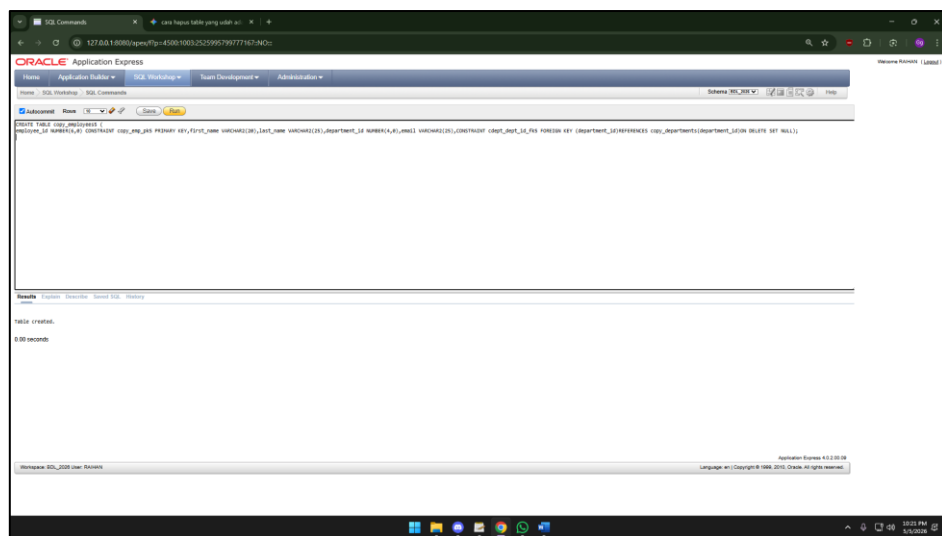
Penjelasan:

Perintah `CREATE TABLE` digunakan untuk membuat tabel `copy_employees4`. Pada definisinya, `employee_id` ditetapkan sebagai `PRIMARY KEY`, sedangkan `department_id` dijadikan `FOREIGN KEY` yang mengacu ke tabel `copy_departments` dengan aturan `ON DELETE CASCADE`. Hasilnya, jika data pada tabel `copy_departments` dihapus, maka data karyawan yang terkait juga akan ikut terhapus secara otomatis.

16. Query:

```
CREATE TABLE copy_employees5 (  
  employee_id NUMBER(6,0) CONSTRAINT copy_emp_pk5  
  PRIMARY KEY, first_name VARCHAR2(20), last_name  
  VARCHAR2(25), department_id NUMBER(4,0), email  
  VARCHAR2(25), CONSTRAINT cdept_dept_id_fk5 FOREIGN  
  KEY (department_id) REFERENCES  
  copy_departments(department_id) ON DELETE SET NULL);
```

Hasil:



Gambar 9.16 Tampilan *create* tabel `copy_employees` dengan ON DELETE SET NULL

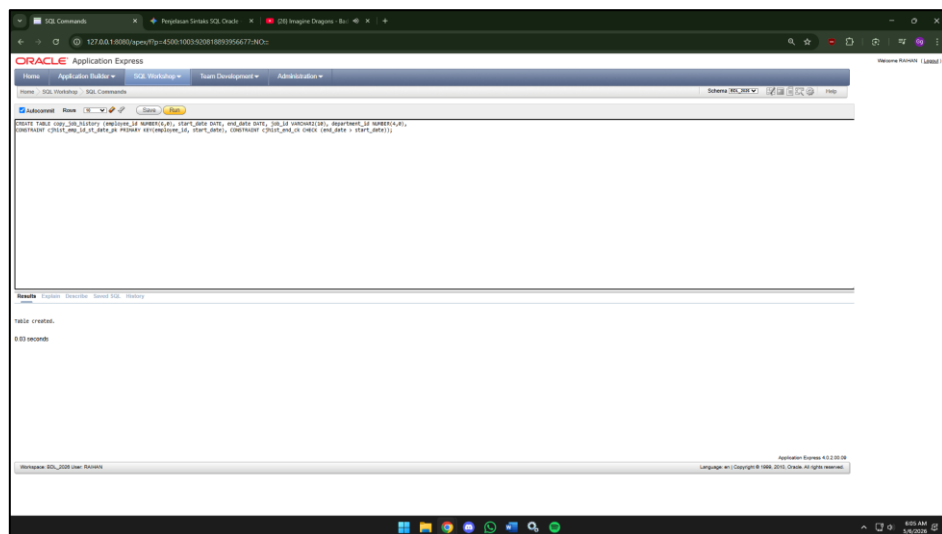
Penjelasan:

Perintah `CREATE TABLE` digunakan untuk membuat tabel `copy_employees4`. Pada definisinya, `employee_id` ditetapkan sebagai `PRIMARY KEY`, sedangkan `department_id` dijadikan `FOREIGN KEY` yang mengacu ke tabel `copy_departments` dengan aturan `ON DELETE SET NULL`. Hasilnya, jika data departemen dihapus, maka nilai `department_id` pada data karyawan akan berubah menjadi `NULL`.

17. Query:

```
CREATE TABLE copy_job_history (employee_id  
NUMBER(6,0), start_date DATE, end_date DATE, job_id  
VARCHAR2(10), department_id NUMBER(4,0),  
CONSTRAINT cjhist_emp_id_st_date_pk PRIMARY  
KEY(employee_id, start_date), CONSTRAINT  
cjhist_end_ck CHECK (end_date > start_date));
```

Hasil:



Gambar 9.17 Tampilan *create* tabel *copy_job_history* dengan
PRIMARY KEY gabungan dan CHECK CONSTRAINT

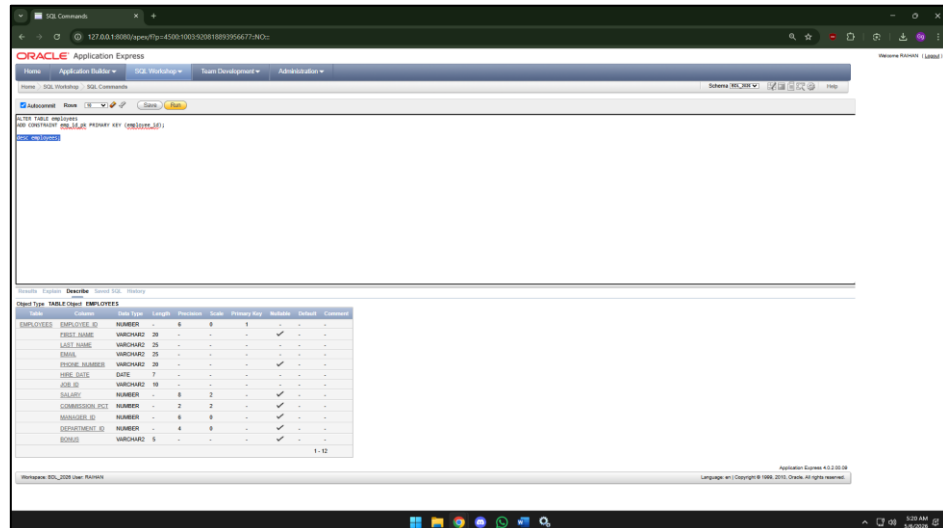
Penjelasan:

Perintah `CREATE TABLE` digunakan untuk membuat tabel *copy_job_history*. Pada definisinya, kombinasi *employee_id* dan *start_date* ditetapkan sebagai PRIMARY KEY, serta ditambahkan CHECK agar nilai *end_date* harus lebih besar dari *start_date*. Hasilnya, tabel berhasil dibuat dengan kunci utama gabungan dan aturan validasi pada tanggal kerja.

18. Query:

```
ALTER TABLE employees  
ADD CONSTRAINT emp_id_pk PRIMARY KEY (employee_id);
```

Hasil:



Gambar 9.18 Tampilan penambahan PRIMARY KEY di kolom `employee_id`

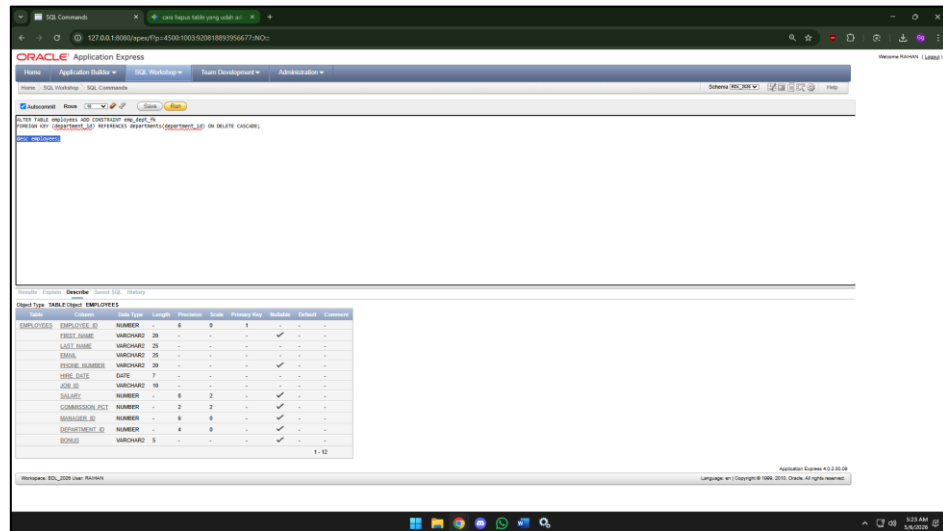
Penjelasan:

Perintah `ALTER TABLE` digunakan untuk menambahkan `CONSTRAINT PRIMARY KEY` bernama `emp_id_pk` pada tabel `employees`. Dalam definisinya, kolom `employee_id` dijadikan kunci utama sehingga setiap data karyawan memiliki identitas yang unik. Hasilnya, tabel `employees` kini memiliki `PRIMARY KEY` pada kolom tersebut.

19. Query:

```
ALTER TABLE employees ADD CONSTRAINT emp_dept_fk  
FOREIGN KEY (department_id) REFERENCES departments  
(department_id) ON DELETE CASCADE;
```

Hasil:



Gambar 9.19 Tampilan penambahan FOREIGN KEY pada tabel employees dengan ON DELETE CASCADE

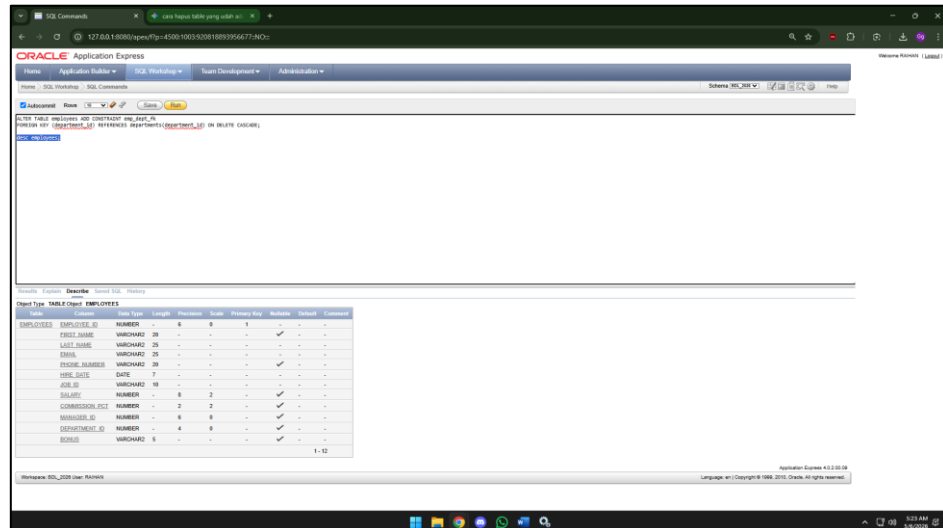
Penjelasan:

Perintah `ALTER TABLE` digunakan untuk menambahkan FOREIGN KEY bernama `emp_dept_fk` pada tabel `employees`. Kolom `department_id` dihubungkan dengan kolom `department_id` pada tabel `departments`, serta diberi aturan `ON DELETE CASCADE`. Hasilnya, jika data departemen dihapus, maka data karyawan yang terkait juga akan ikut terhapus.

20. Query:

```
ALTER TABLE employees MODIFY (email CONSTRAINT  
emp_email_nn NOT NULL)
```

Hasil:



Gambar 9.20 Tampilan perubahan kolom email menjadi NOT NULL

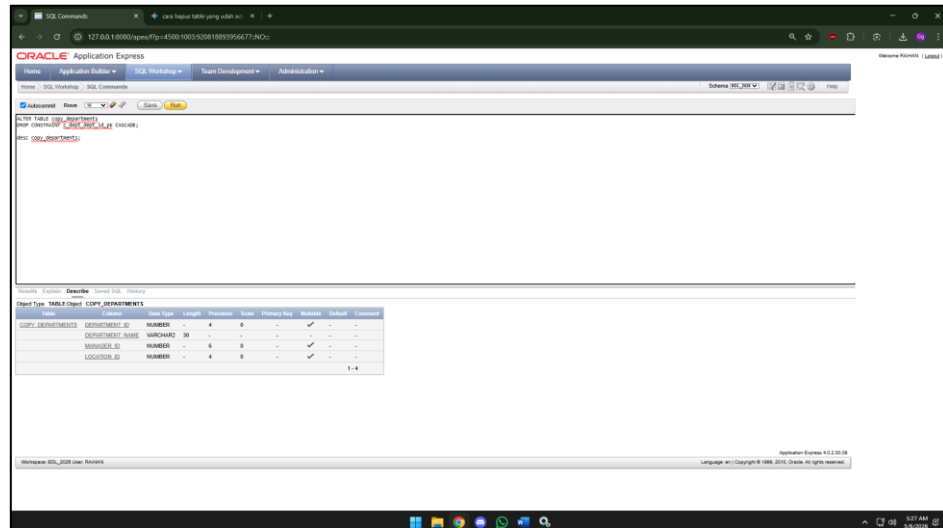
Penjelasan:

Perintah `ALTER TABLE` digunakan untuk mengubah kolom email pada tabel `employees` agar menjadi wajib diisi. Pada prosesnya, ditambahkan `CONSTRAINT NOT NULL` bernama `emp_email_nn` pada kolom tersebut. Hasilnya, tabel `employees` tidak akan menerima data karyawan dengan nilai email yang kosong.

21. Query:

```
ALTER TABLE copy_departments  
DROP CONSTRAINT c_dept_dept_id_pk CASCADE;
```

Hasil:



Gambar 9.21 Tampilan penghapusan CONSTRAINT

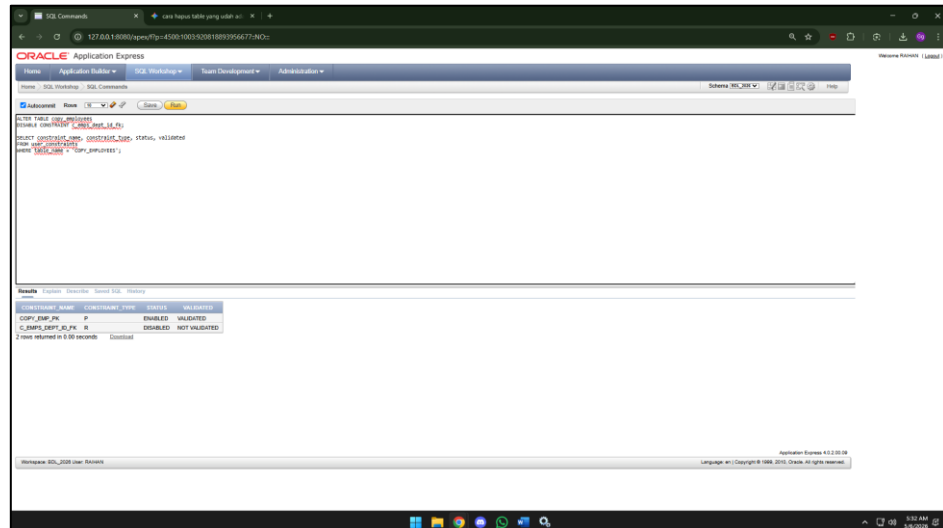
Penjelasan:

Perintah `ALTER TABLE` digunakan untuk menghapus CONSTRAINT `c_dept_dept_id_pk` pada tabel `copy_departments`. Pada prosesnya, digunakan opsi `CASCADE` agar CONSTRAINT lain yang bergantung pada PRIMARY KEY tersebut ikut terhapus. Hasilnya, tabel `copy_departments` tidak lagi memiliki CONSTRAINT tersebut beserta ketergantungannya.

22. Query:

```
ALTER TABLE copy_employees  
DISABLE CONSTRAINT c_emps_dept_id_fk;
```

Hasil:



Gambar 9.22 Tampilan penonaktifan CONSTRAINT

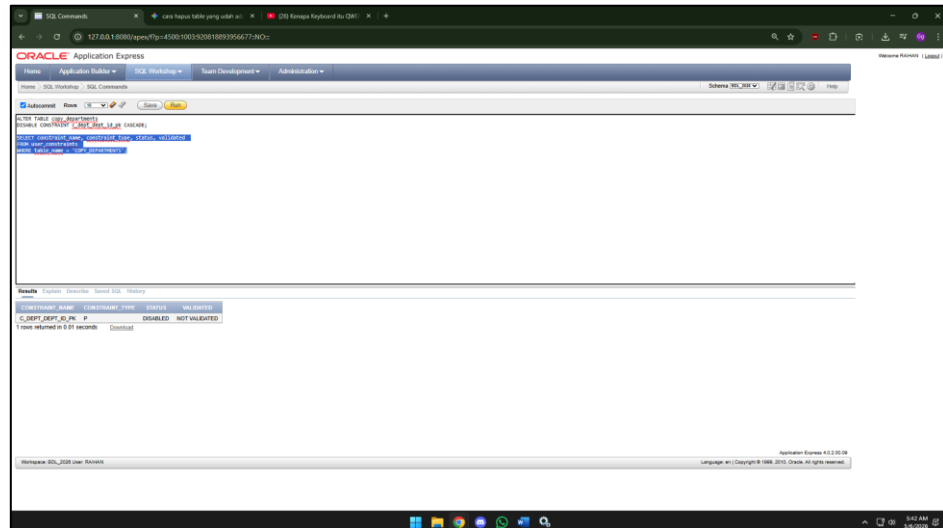
Penjelasan:

Perintah `ALTER TABLE` digunakan untuk menonaktifkan CONSTRAINT `c_emp_dept_id_fk` pada tabel `copy_employees`. Dengan perintah ini, aturan FOREIGN KEY tersebut tidak berlaku sementara tanpa dihapus dari tabel. Hasilnya, CONSTRAINT tersebut berhasil dinonaktifkan dan dapat diaktifkan kembali jika diperlukan.

23. Query:

```
ALTER TABLE copy_departments  
DISABLE CONSTRAINT c_dept_dept_id_pk CASCADE;
```

Hasil:



Gambar 9.23 Tampilan penonaktifan CONSTRAINT dengan CASCADE

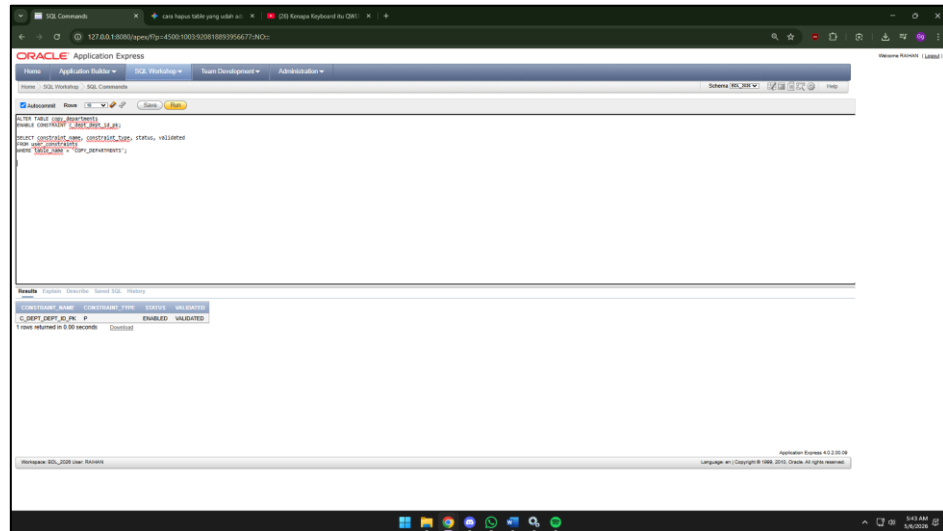
Penjelasan:

Perintah `ALTER TABLE` digunakan untuk menonaktifkan CONSTRAINT `c_dept_dept_id_pk` pada tabel `copy_departments`. Opsi `CASCADE` digunakan agar CONSTRAINT lain yang bergantung padanya ikut dinonaktifkan. Hasilnya, CONSTRAINT tersebut beserta aturan terkait berhasil dinonaktifkan sementara.

24. Query:

```
ALTER TABLE copy_departments  
ENABLE CONSTRAINT c_dept_dept_id_pk;
```

Hasil:



Gambar 9.24 Tampilan pengaktifan kembali CONSTRAINT

c_dept_dept_id_pk

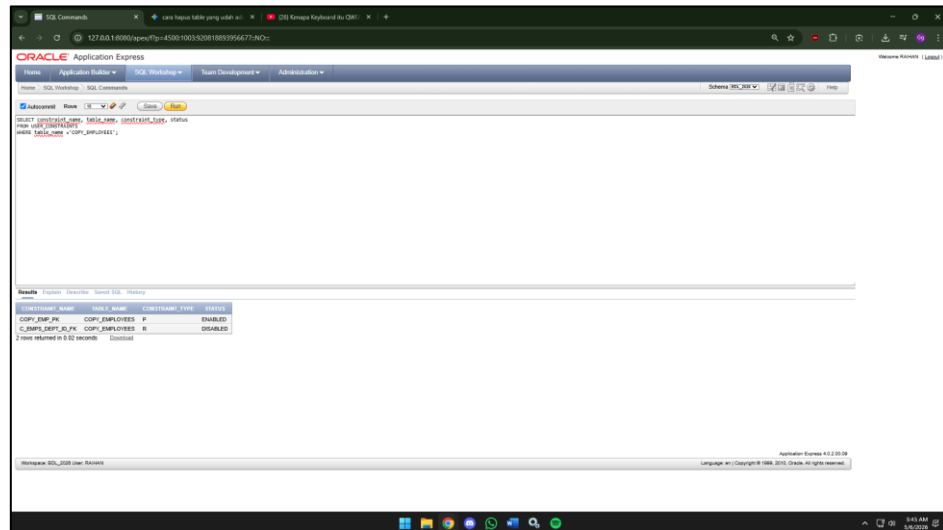
Penjelasan:

Perintah `ALTER TABLE` digunakan untuk mengaktifkan kembali CONSTRAINT `c_dept_dept_id_pk` pada tabel `copy_departments`. Dengan perintah ini, aturan CONSTRAINT tersebut kembali diberlakukan pada tabel. Hasilnya, CONSTRAINT tersebut berhasil aktif kembali dan dapat digunakan seperti semula.

25. Query:

```
SELECT constraint_name, table_name,  
constraint_type, status  
FROM USER_CONSTRAINTS  
WHERE table_name = 'COPY_EMPLOYEES';
```

Hasil:



Gambar 9.25 Tampilan untuk melihat CONSTRAINT pada tabel
COPY_EMPLOYEES

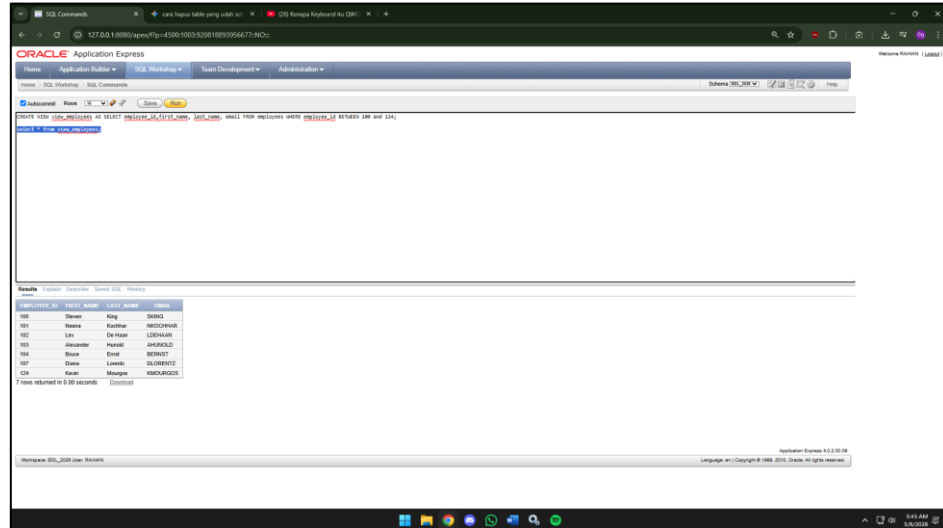
Penjelasan:

Perintah SELECT digunakan untuk menampilkan informasi CONSTRAINT pada tabel COPY_EMPLOYEES. Data yang diambil meliputi nama CONSTRAINT, nama tabel, jenis, serta statusnya dari USER_CONSTRAINTS. Hasilnya, daftar CONSTRAINT pada tabel tersebut dapat diketahui.

26. Query:

```
CREATE VIEW view_employees AS SELECT  
employee_id, first_name, last_name, email FROM  
employees WHERE employee_id BETWEEN 100 and 124;
```

Hasil:



The screenshot shows the Oracle SQL Developer interface. The SQL Command window contains the following query:

```
CREATE VIEW view_employees AS SELECT  
employee_id, first_name, last_name, email FROM  
employees WHERE employee_id BETWEEN 100 and 124;
```

The Results window displays the following data:

EMPLOYEE_ID	FIRST_NAME	LAST_NAME	EMAIL
100	Steven	King	SKING
101	Neena	Kochhar	NKOCHHAR
102	Lex	De Haan	LDEHAAN
103	Alexander	Toole	ATOOLE
104	Bruce	Ernst	BERNST
105	Diana	Lavigne	DLAVIGNE
124	Renske	Sharma	RSHARMA

7 rows returned in 0.00 seconds

Gambar 9.26 Tampilan pembuatan VIEW view_employees

Penjelasan:

Perintah `CREATE VIEW` digunakan untuk membuat `view_employees` dari tabel `employees`. View ini menampilkan kolom untuk data karyawan dengan `employee_id` antara 100 hingga 124. Hasilnya, `view_employees` berhasil dibuat untuk menampilkan data sesuai rentang tersebut.

27. Query:

```
SELECT * FROM view_employees;
```

Hasil:

EMPLOYEE_ID	FIRST_NAME	LAST_NAME	EMAIL	PHONE_NUMBER
106	Steven	King	SKING	555.1234
107	Neena	Kochhar	NKOCHHAR	555.1234
108	Lex	De Haan	LDEHAAN	555.1234
109	Alexander	Russell	ARUSSEL	555.1234
110	Bruce	Ernst	BENERT	555.1234
111	Dena	Lewis	DLLEWIS	555.1234
112	Kevin	Moura	KMOURA	555.1234

Gambar 9.27 Tampilan seluruh data dari view_employees

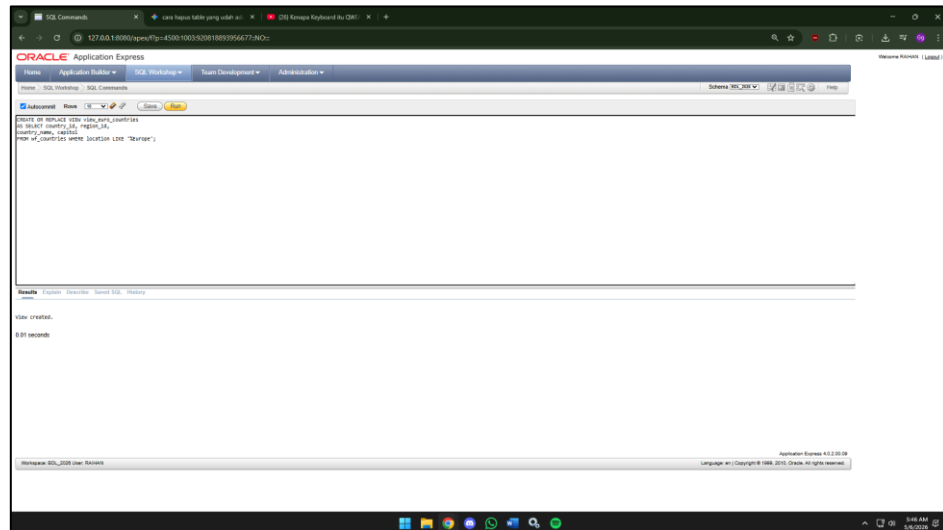
Penjelasan:

Perintah tersebut digunakan untuk menampilkan seluruh data dari view_employees. Perintah SELECT ini mengambil semua kolom dan baris yang tersedia pada VIEW tersebut. Hasil akhirnya adalah data karyawan pada view_employees berhasil ditampilkan.

28. Query:

```
CREATE OR REPLACE VIEW view_euro_countries
AS SELECT country_id, region_id,
country_name, capitol
FROM wf_countries WHERE location LIKE '%Europe';
```

Hasil:



Gambar 9.28 Tampilan pembuatan view_euro_countries dari tabel wf_countries

Penjelasan:

Perintah `CREATE OR REPLACE VIEW` digunakan untuk membuat atau memperbarui view_euro_countries dari tabel wf_countries. View ini menampilkan kolom untuk data negara dengan lokasi “Europe”. Hasilnya, view_euro_countries berhasil dibuat atau diperbarui sesuai kriteria tersebut.

29. Query:

```
SELECT * FROM view_euro_countries
ORDER BY country_name;
```

Hasil:

The screenshot displays the Oracle Application Express (APEX) interface. At the top, the navigation bar includes 'Home', 'Application Builder', 'SQL Workshop', 'Team Development', and 'Administration'. The 'SQL Workshop' tab is selected, showing a 'Schema TO_TO' view. Below the navigation bar, there's a 'Substatement' section with a 'Run' button. The main area displays the results of the SQL query: 'SELECT * FROM SYS.EMPLOYEE'. The results are shown in a table with columns: EMPLOYEE_ID, EMPLOYEE_NAME, and EMPLOYEE_JOB. The table contains 10 rows of data, including employees like 'Scott', 'Tjerry', 'Ramesh', 'Suresh', 'Priya', 'Neha', 'Ravi', 'Suresh', 'Priya', and 'Neha'. The bottom status bar shows 'Workspace: SYS_2020 User: SYS' and 'Application Express 4.2.0.10'.

Gambar 9.29 Tampilan seluruh data dari `view_euro_countries`

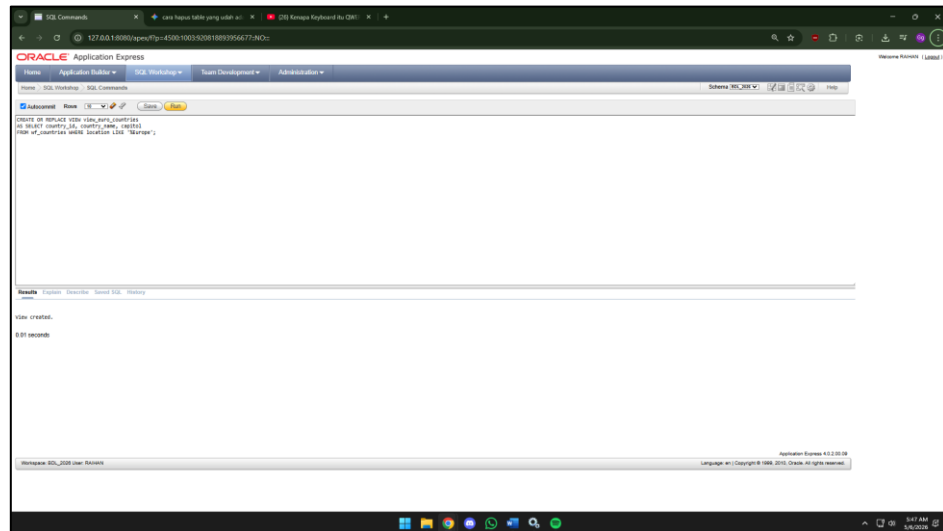
Penjelasan:

Perintah `SELECT` digunakan untuk menampilkan seluruh data dari `view_euro_countries`. Data yang dihasilkan kemudian diurutkan berdasarkan kolom `country_name` secara *ascending*. Hasilnya, daftar negara ditampilkan secara berurutan sesuai nama negara.

30. Query:

```
CREATE OR REPLACE VIEW view_euro_countries  
AS SELECT country_id, country_name, capitol  
FROM wf_countries WHERE location LIKE '%Europe';
```

Hasil:



Gambar 9.30 Tampilan pembuatan view_euro_countries dengan kolom country_id, country_name, dan capitol

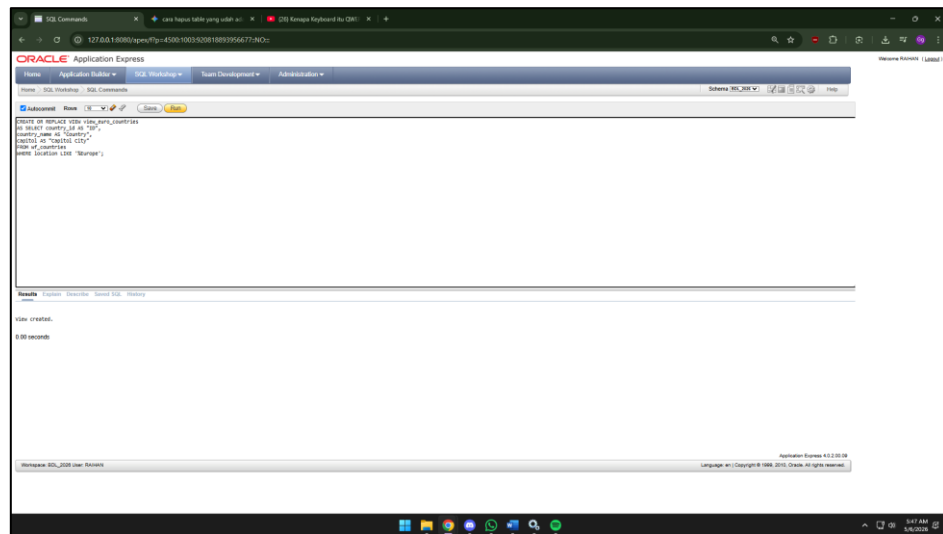
Penjelasan:

Perintah CREATE OR REPLACE VIEW digunakan untuk membuat atau memperbarui view_euro_countries dari tabel wf_countries. View ini hanya menampilkan kolom country_id, country_name, dan capitol untuk negara dengan lokasi “Europe”. Hasilnya, view_euro_countries berhasil dibuat atau diperbarui dengan jumlah kolom yang lebih sederhana.

31. Query:

```
CREATE OR REPLACE VIEW view_euro_countries
AS SELECT country_id AS "ID",
country_name AS "Country",
capitol AS "Capitol City"
FROM wf_countries
WHERE location LIKE '%Europe';
```

Hasil:



Gambar 9.31 Tampilan *create* view_euro_countries dengan alias

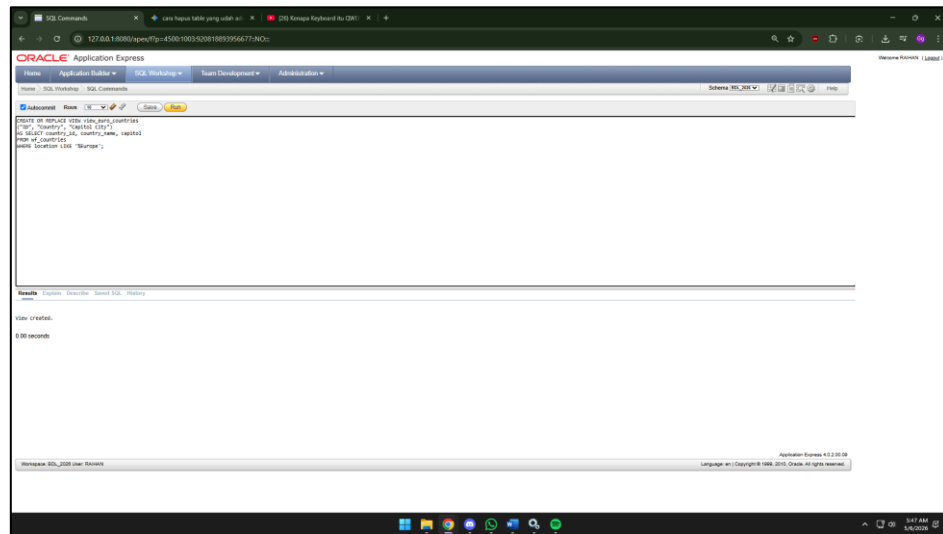
Penjelasan:

Perintah `CREATE OR REPLACE VIEW` digunakan untuk membuat atau memperbarui `view_euro_countries` dari tabel `wf_countries`. View ini menampilkan `country_id`, `country_name`, dan `capitol` dengan alias kolom “ID”, “Country”, dan “Capitol City” untuk negara yang berlokasi “Europe”. Hasilnya, `view_euro_countries` berhasil dibuat atau diperbarui dengan nama kolom yang lebih mudah dipahami.

32. Query:

```
CREATE OR REPLACE VIEW view_euro_countries
("ID", "Country", "Capitol City")
AS SELECT country_id, country_name, capitol
FROM wf_countries
WHERE location LIKE '%Europe';
```

Hasil:



Gambar 9.32 Tampilan *create* view_euro_countries dengan penamaan kolom “ID”, “Country”, dan “Capitol City”

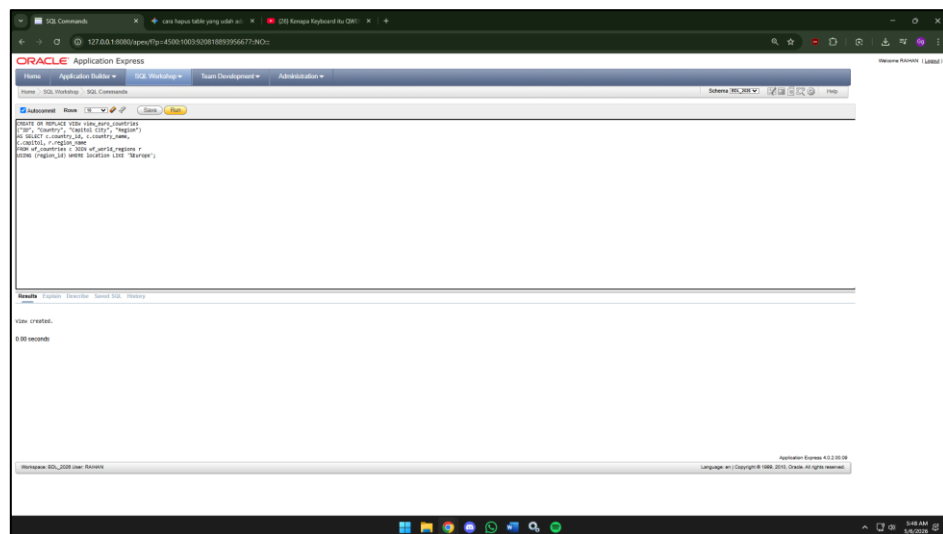
Penjelasan:

Perintah tersebut digunakan untuk membuat atau mengganti VIEW bernama view_euro_countries dengan nama kolom alias, yaitu “ID”, “Country”, dan “Capitol City”. Perintah ini mengambil data country_id, country_name, dan capitol dari tabel wf_countries untuk negara yang memiliki lokasi “Europe”. Hasil akhirnya adalah view_euro_countries berhasil dibuat atau diperbarui dengan nama kolom yang ditentukan langsung pada bagian VIEW.

33. Query:

```
CREATE OR REPLACE VIEW view_euro_countries
("ID", "Country", "Capitol City", "Region")
AS SELECT c.country_id, c.country_name,
c.capitol, r.region_name
FROM wf_countries c JOIN wf_world_regions r
USING (region_id) WHERE location LIKE '%Europe';
```

Hasil:



Gambar 9.33 Tampilan *create* view_euro_countries dengan JOIN

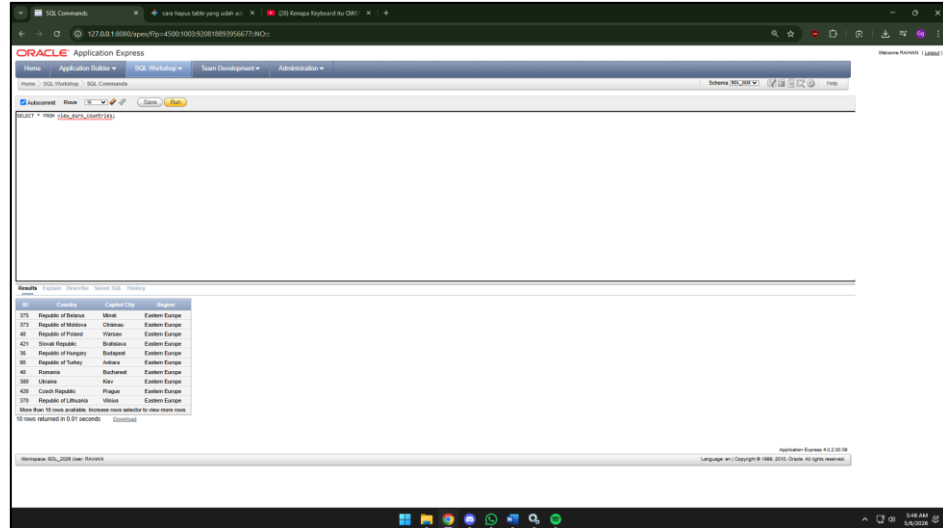
Penjelasan:

Perintah CREATE OR REPLACE VIEW digunakan untuk membuat atau memperbarui view_euro_countries dengan kolom “ID”, “Country”, “Capitol City”, dan “Region”. Data diambil dari tabel wf_countries lalu digabungkan dengan wf_world_regions melalui region_id untuk menampilkan negara yang berlokasi “Europe”. Hasilnya, view tersebut berisi informasi negara, ibu kota, dan nama region.

34. Query:

```
SELECT * FROM view_euro_countries;
```

Hasil:



ID	Country	Capital	Region
373	Republic of Romania	Bucharest	Eastern Europe
373	Republic of Moldova	Chişinău	Eastern Europe
461	Republic of Poland	Warsaw	Eastern Europe
427	Slovak Republic	Bratislava	Eastern Europe
36	Republic of Hungary	Budapest	Eastern Europe
95	Republic of Turkey	Ankara	Eastern Europe
461	Romania	Bucharest	Eastern Europe
385	Ukraine	Kiev	Eastern Europe
429	Czech Republic	Prague	Eastern Europe
373	Republic of Lithuania	Vilnius	Eastern Europe

Gambar 9.34 Tampilan seluruh data dari view_euro_countries

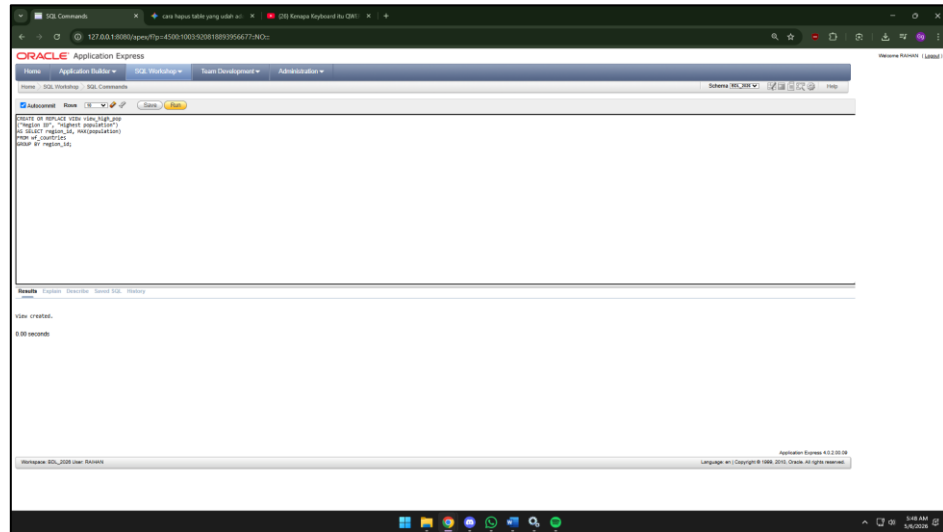
Penjelasan:

Perintah tersebut digunakan untuk menampilkan seluruh data dari view_euro_countries. Perintah SELECT ini mengambil semua kolom dan baris yang tersedia pada VIEW tersebut. Hasil akhirnya adalah data negara Eropa beserta ID, nama negara, ibu kota, dan region berhasil ditampilkan.

35. Query:

```
CREATE OR REPLACE VIEW view_high_pop  
("Region ID", "Highest population")  
AS SELECT region_id, MAX(population)  
FROM wf_countries  
GROUP BY region_id;
```

Hasil:



Gambar 9.35 Tampilan *create* VIEW view_high_pop berdasarkan populasi tertinggi tiap *region*

Penjelasan:

Perintah CREATE OR REPLACE VIEW digunakan untuk membuat atau memperbarui view_high_pop dengan kolom “Region ID” dan “Highest population”. Data diambil dari tabel wf_countries dengan mengambil nilai populasi tertinggi untuk setiap region_id. Hasilnya, view tersebut menampilkan populasi terbesar pada masing-masing region.

36. Query:

```
SELECT * FROM view_high_pop;
```

Hasil:

Region ID	Highest population
5	10007022
9	29294802
11	131008715
13	91744625
14	74777881
15	70887047
17	62088051
18	44107107
21	28844215
25	11302020

Gambar 9.36 Tampilan seluruh data dari view_high_pop

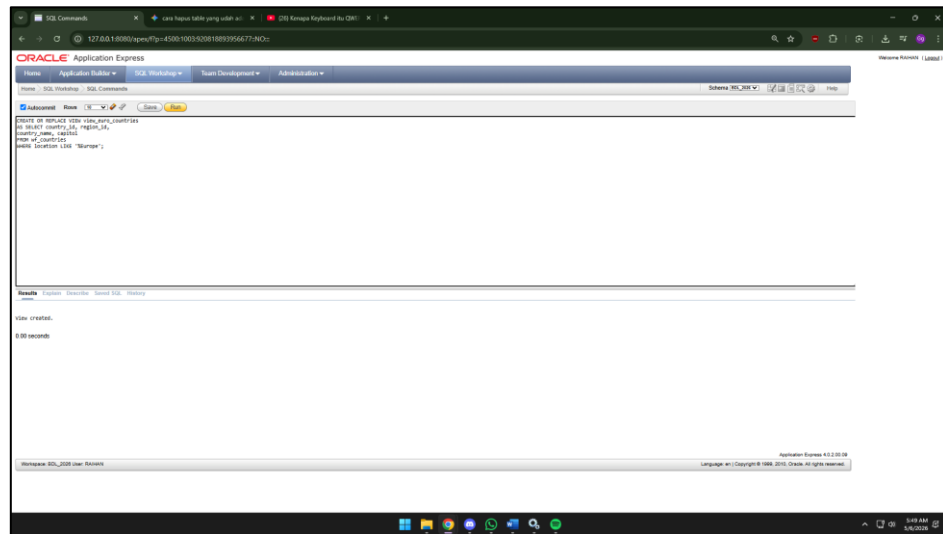
Penjelasan:

Perintah tersebut digunakan untuk menampilkan seluruh data dari view_high_pop. Perintah SELECT ini mengambil semua kolom dan baris yang berisi “Region ID” serta “Highest population”. Hasil akhirnya adalah data populasi tertinggi pada tiap region berhasil ditampilkan.

37. Query:

```
CREATE OR REPLACE VIEW view_euro_countries
AS SELECT country_id, region_id,
country_name, capitol
FROM wf_countries
WHERE location LIKE '%Europe';
```

Hasil:



Gambar 9.37 Tampilan *create* view_euro_countries

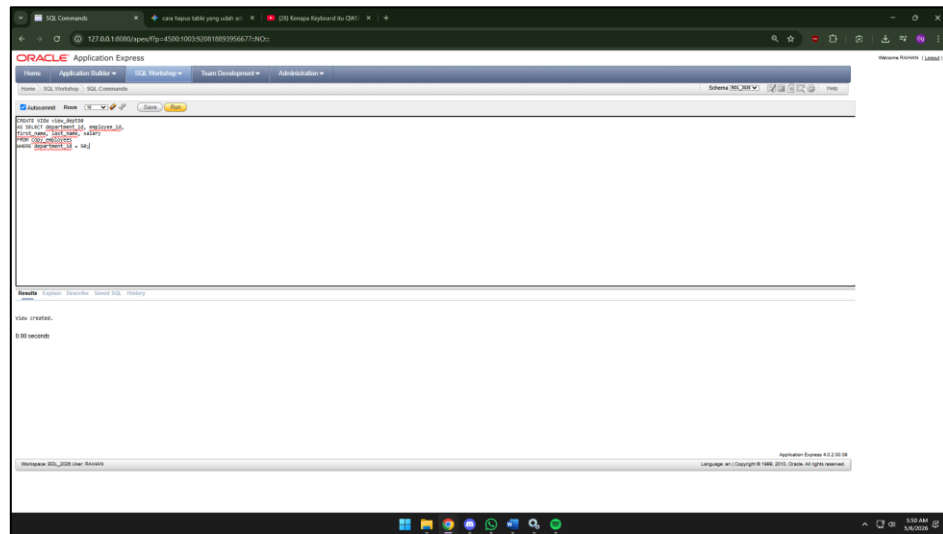
Penjelasan:

Perintah `CREATE OR REPLACE VIEW` digunakan untuk membuat atau memperbarui `view_euro_countries` dari tabel `wf_countries`. View ini menampilkan kolom `country_id`, `region_id`, `country_name`, dan `capitol` untuk negara yang berlokasi “Europe”. Hasilnya, `view_euro_countries` berhasil dibuat atau diperbarui sesuai kriteria tersebut.

38. Query:

```
CREATE VIEW view_dept50
AS SELECT department_id, employee_id,
first_name, last_name, salary
FROM copy_employees
WHERE department_id = 50;
```

Hasil:



Gambar 9.38 Tampilan *create* VIEW *view_dept50* dari tabel *copy_employees*

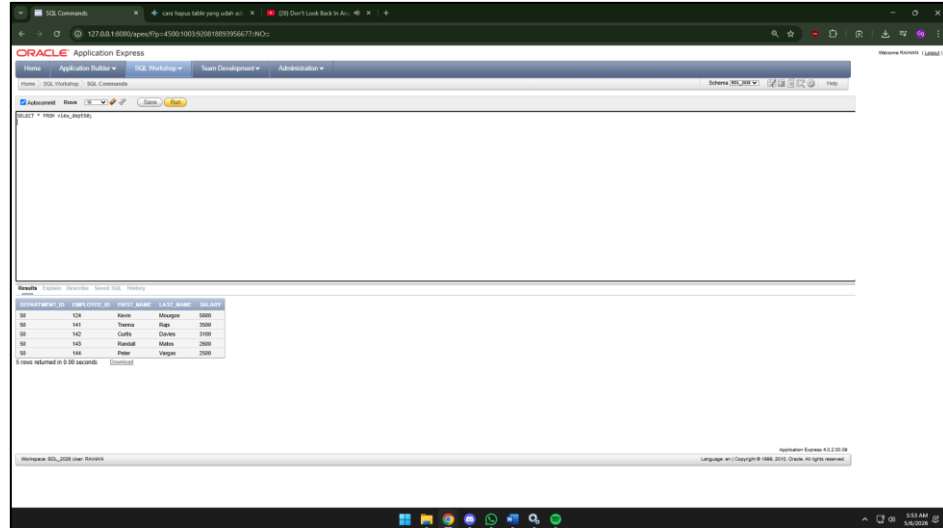
Penjelasan:

Perintah `CREATE VIEW` digunakan untuk membuat *view_dept50* dari tabel *copy_employees*. View ini menampilkan kolom *department_id*, *employee_id*, *first_name*, *last_name*, dan *salary* untuk karyawan dengan *department_id* = 50. Hasilnya, *view_dept50* berhasil dibuat untuk menampilkan data karyawan pada departemen tersebut.

39. Query:

```
SELECT * FROM view_dept50;
```

Hasil:



DEPARTMENT_ID	EMPLOYEE_ID	NAME	JOB	MANAGER	DATE_HIRED	SALARY
50	101	Steven	Manager		2007-06-07	24000
50	102	Neena	Analyst	101	2007-06-07	3000
50	103	Lex	Analyst	101	2007-06-07	3000
50	104	Patel	Analyst	101	2007-06-07	3000
50	105	Verges	Analyst	101	2007-06-07	3000

Gambar 9.39 Tampilan seluruh data dari view_dept50

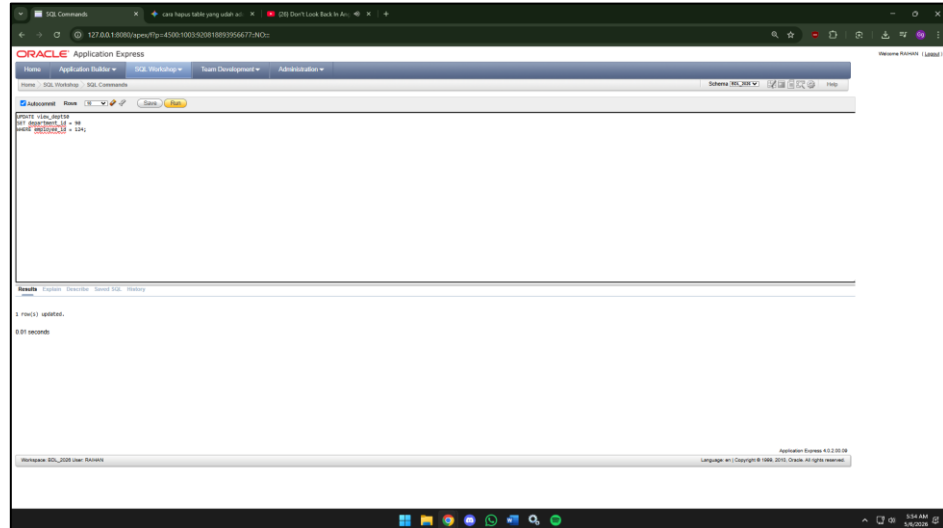
Penjelasan:

Perintah tersebut digunakan untuk menampilkan seluruh data dari view_dept50. Perintah SELECT ini mengambil semua kolom dan baris yang berisi data karyawan pada department_id 50. Hasil akhirnya adalah data karyawan dari departemen 50 berhasil ditampilkan.

40. Query:

```
UPDATE view_dept50  
SET department_id = 90  
WHERE employee_id = 124;
```

Hasil:



Gambar 9.40 Tampilan perubahan `department_id` pada `view_dept50` untuk `employee_id = 124`

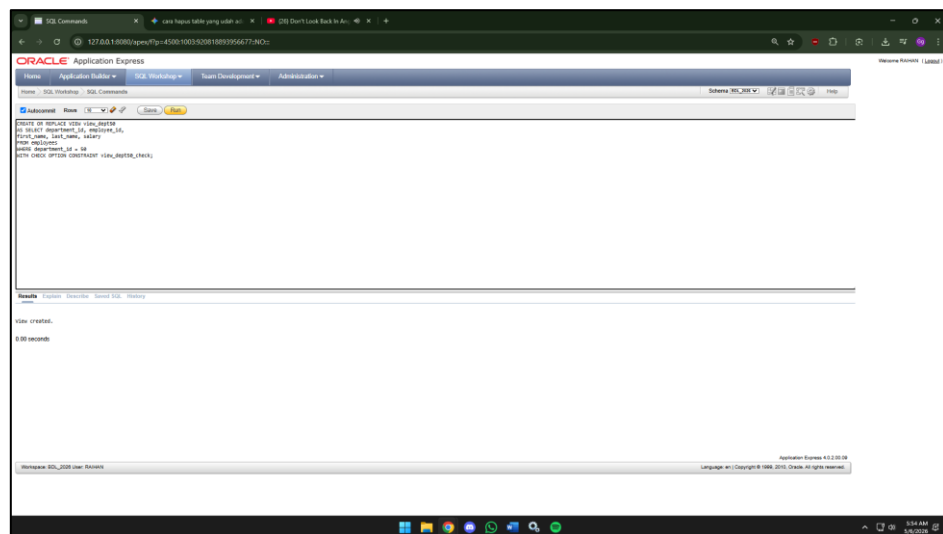
Penjelasan:

Perintah `UPDATE` digunakan untuk mengubah nilai `department_id` pada `view_dept50` untuk karyawan dengan `employee_id = 124`. Nilai `department_id` diubah menjadi 90, sehingga perubahan ini juga berdampak pada tabel dasar `copy_employees`. Akibatnya, data karyawan tersebut tidak lagi muncul di `view_dept50` karena view hanya menampilkan data dengan `department_id 50`.

41. Query:

```
CREATE OR REPLACE VIEW view_dept50
AS SELECT department_id, employee_id,
first_name, last_name, salary
FROM employees
WHERE department_id = 50
WITH CHECK OPTION CONSTRAINT view_dept50_check;
```

Hasil:



Gambar 9.41 Tampilan *create* view_dept50 dengan CHECK OPTION CONSTRAINT

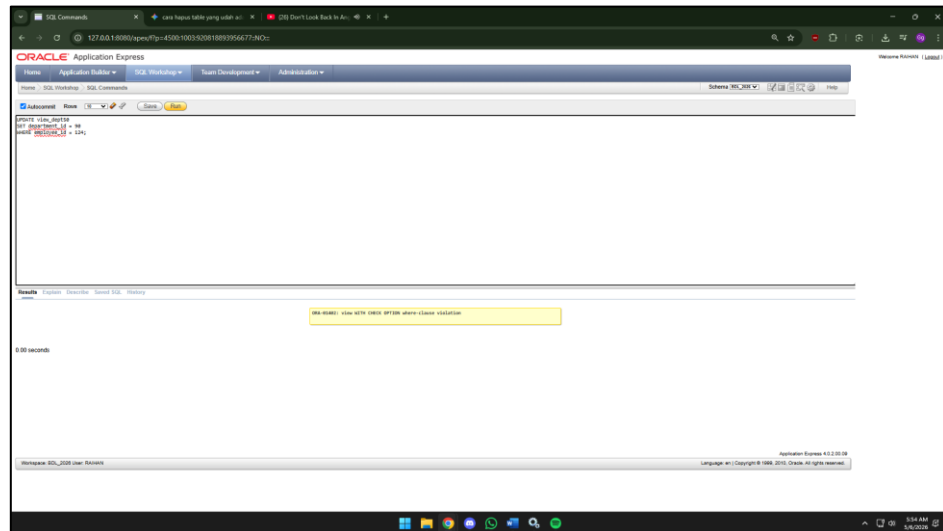
Penjelasan:

Perintah `CREATE OR REPLACE VIEW` digunakan untuk membuat ulang view_dept50 dari tabel employees yang hanya menampilkan data karyawan dengan `department_id = 50`. Pada definisinya, digunakan `WITH CHECK OPTION` agar setiap perubahan data melalui view tetap harus memenuhi kondisi tersebut. Hasilnya, view_dept50 berhasil dibuat dengan aturan validasi view_dept50_check.

42. Query:

```
UPDATE view_dept50  
SET department_id = 90  
WHERE employee_id = 124;
```

Hasil:



Gambar 9.42 Tampilan hasil *error* perubahan `department_id` pada `view_dept50` karena `CHECK OPTION CONSTRAINT`

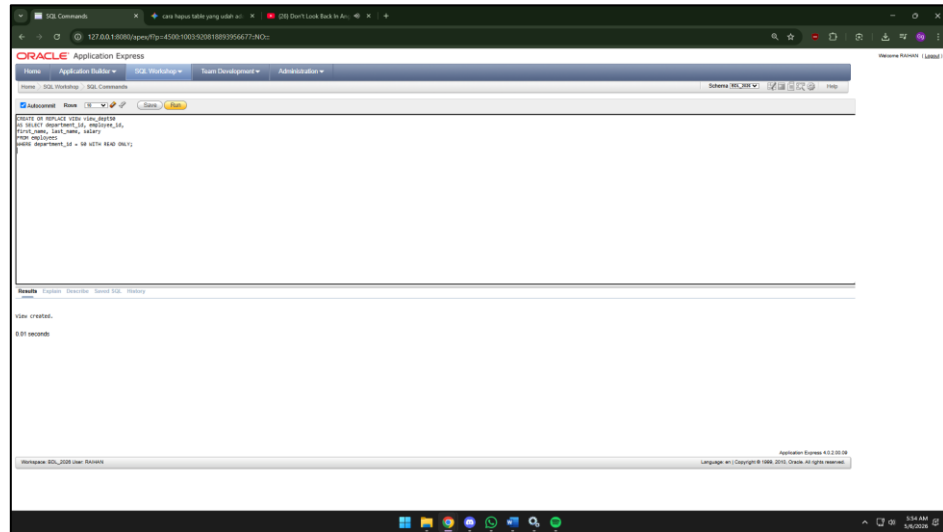
Penjelasan:

Query tersebut digunakan untuk mengubah `department_id` menjadi 90 pada `view_dept50` untuk `employee_id = 124`. Perintah `UPDATE` ini tidak dapat dijalankan karena `view_dept50` memiliki `WITH CHECK OPTION` yang hanya mengizinkan data dengan `department_id = 50`. Hasil akhirnya adalah perubahan ditolak karena nilai `department_id = 90` tidak sesuai dengan aturan `VIEW`.

43. Query:

```
CREATE OR REPLACE VIEW view_dept50
AS SELECT department_id, employee_id,
first_name, last_name, salary
FROM employees
WHERE department_id = 50 WITH READ ONLY;
```

Hasil:



Gambar 9.43 Tampilan hasil pembuatan view_dept50 dengan READ ONLY CONSTRAINT

Penjelasan:

Perintah tersebut digunakan untuk membuat ulang view_dept50 dari tabel employees dengan data karyawan yang memiliki department_id = 50. Perintah ini menggunakan WITH READ ONLY agar data pada VIEW hanya dapat ditampilkan dan tidak dapat diubah melalui perintah INSERT, UPDATE, atau DELETE. Hasil akhirnya adalah view_dept50 berhasil dibuat sebagai VIEW yang bersifat hanya baca.

44. Query:

```
SELECT e.last_name, e.salary,  
e.department_id, d.maxsal  
FROM employees e,  
(SELECT department_id, max(salary) maxsal  
FROM employees  
GROUP BY department_id) d  
WHERE e.department_id = d.department_id  
AND e.salary = d.maxsal;
```

Hasil:

The screenshot shows the Oracle SQL Developer interface. The query window contains the SQL query from the previous block. The Results window displays the output of the query, which lists the last name, salary, and department ID for the highest paid employee in each department. The data is as follows:

last_name	salary	department_id
King	24000	90
Plurien	13000	20
Higgins	12000	100
Jones	11000	90
Blouin	9000	90
Williams	6000	10
Harrell	5000	90

Gambar 9.44 Tampilan karyawan dengan gaji tertinggi pada setiap departemen

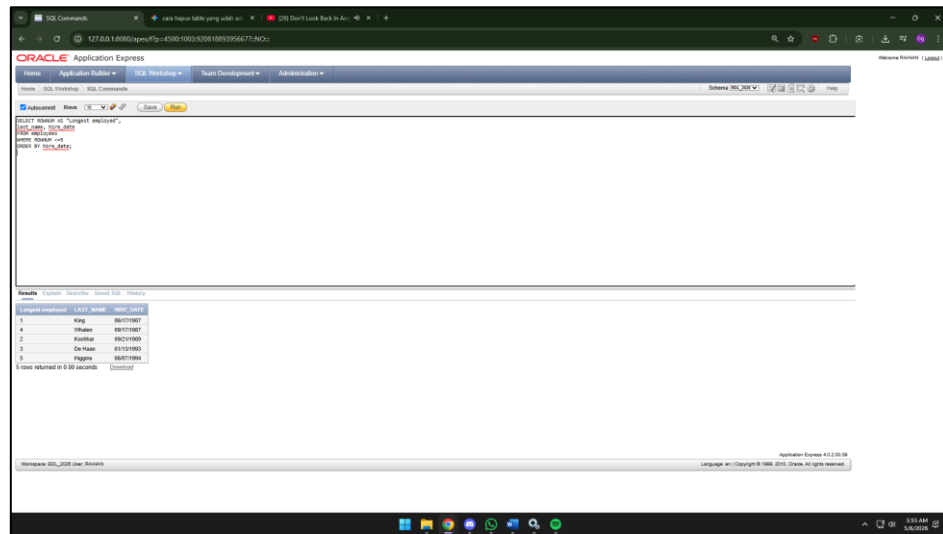
Penjelasan:

Query tersebut digunakan untuk menampilkan data karyawan yang memiliki gaji tertinggi pada setiap departemen. Perintah SELECT ini mengambil last_name, salary, department_id, dan maxsal dengan membandingkan gaji karyawan terhadap hasil subquery maksimum gaji per departemen. Hasil akhirnya adalah daftar karyawan dengan gaji tertinggi di masing-masing departemen berhasil ditampilkan.

45. Query:

```
SELECT ROWNUM AS "Longest employed",  
last_name, hire_date  
FROM employees  
WHERE ROWNUM <=5  
ORDER BY hire_date;
```

Hasil:



Longest employed	last_name	hire_date
1	King	05/17/1987
2	De Haan	09/17/1987
3	Koltrane	09/21/1988
4	Chen	09/29/1988
5	Baggett	09/30/1988

Gambar 9.45 Tampilan lima data karyawan berdasarkan hire_date

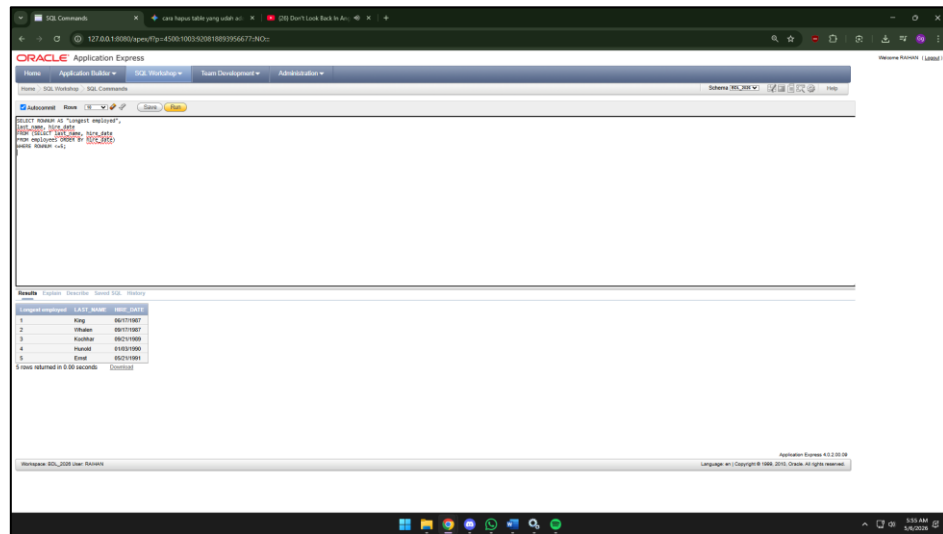
Penjelasan:

Query tersebut digunakan untuk menampilkan lima data karyawan dari tabel employees dengan kolom “Longest employed”, last_name, dan hire_date. Perintah SELECT ini memberi nomor urut melalui ROWNUM, lalu mengurutkan hasil berdasarkan hire_date. Hasil akhirnya adalah lima data karyawan berhasil ditampilkan berdasarkan urutan tanggal mulai bekerja.

46. Query:

```
SELECT ROWNUM AS "Longest employed",  
last_name, hire_date  
FROM (SELECT last_name, hire_date  
FROM employees ORDER BY hire_date)  
WHERE ROWNUM <=5;
```

Hasil:



Longest employed	last_name	hire_date
1	King	05/17/1987
2	White	09/17/1987
3	Kudva	08/21/1988
4	Patel	01/24/1988
5	Deo	08/23/1988

Gambar 9.46 Tampilan lima karyawan paling lama bekerja berdasarkan hire_date

Penjelasan:

Query tersebut digunakan untuk menampilkan lima karyawan yang paling lama bekerja berdasarkan hire_date. Perintah SELECT ini mengurutkan data karyawan dari tanggal masuk paling awal, lalu memberi nomor urut menggunakan ROWNUM. Hasil akhirnya adalah lima karyawan dengan masa kerja paling lama berhasil ditampilkan.

9.4 Latihan

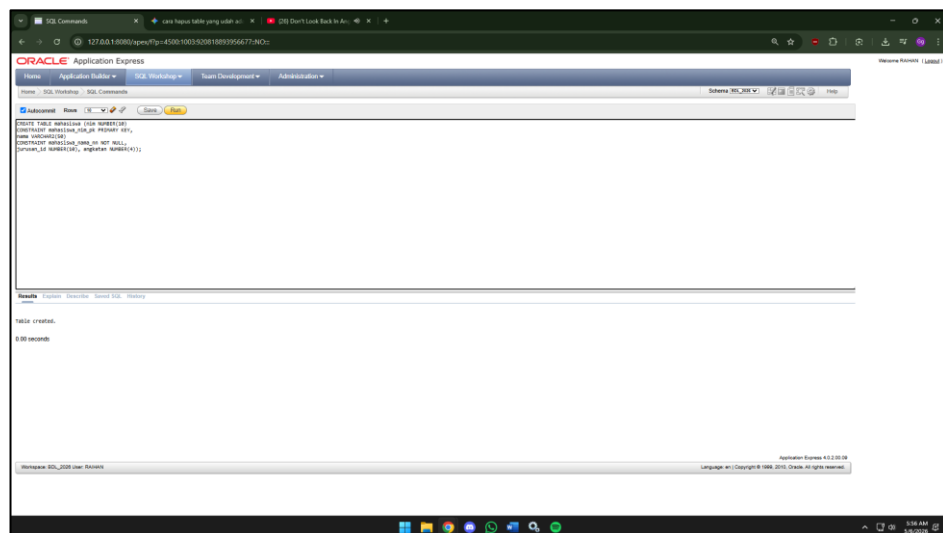
Adapun latihan pada bab ini adalah sebagai berikut:

1. Menggunakan perintah `CREATE TABLE` dengan `CONSTRAINTS` untuk membuat tabel mahasiswa dengan pembatasan tertentu.

Query:

```
CREATE TABLE mahasiswa (nim NUMBER(10)
CONSTRAINT mahasiswa_nim_pk PRIMARY KEY,
nama VARCHAR2(50)
CONSTRAINT mahasiswa_nama_nn NOT NULL,
jurusan_id NUMBER(10), angkatan NUMBER(4));
```

Hasil:



Gambar 9.47 Tampilan *create* tabel mahasiswa dengan `CONSTRAINT` `PRIMARY KEY` dan `NOT NULL`

Penjelasan:

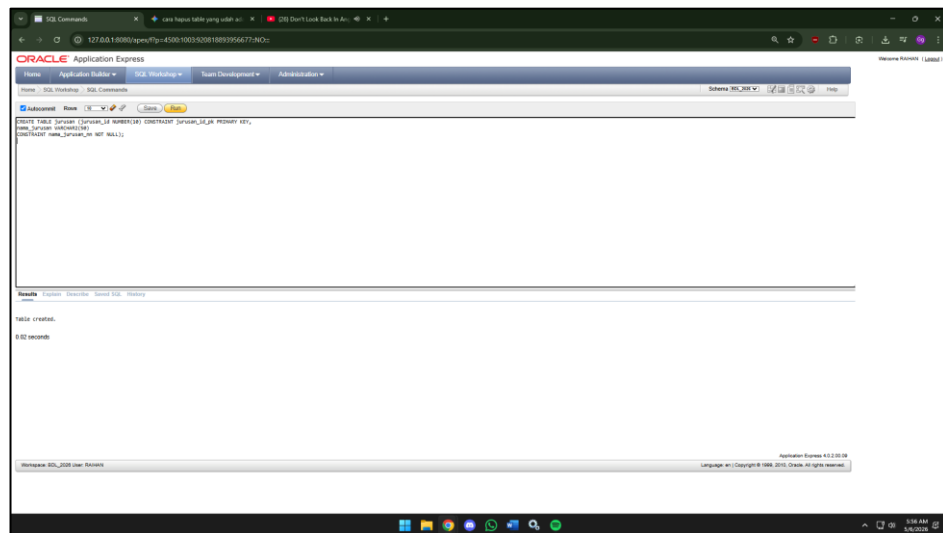
Query tersebut digunakan untuk membuat tabel mahasiswa. Pada perintah `CREATE TABLE`, `nim` ditetapkan sebagai `PRIMARY KEY`, sedangkan `nama` diberi aturan `NOT NULL` sehingga wajib diisi. Hasilnya, tabel mahasiswa berhasil dibuat dengan ketentuan kunci utama dan kolom yang harus terisi.

2. Menggunakan perintah CREATE TABLE dengan CONSTRAINTS untuk membuat tabel jurusan dengan pembatasan tertentu.

Query:

```
CREATE TABLE jurusan (jurusan_id NUMBER(10)
CONSTRAINT jurusan_id_pk PRIMARY KEY,
nama_jurusan VARCHAR2(50)
CONSTRAINT nama_jurusan_nn NOT NULL);
```

Hasil:



Gambar 9.48 Tampilan *create* tabel jurusan dengan CONSTRAINT PRIMARY KEY dan NOT NULL

Penjelasan:

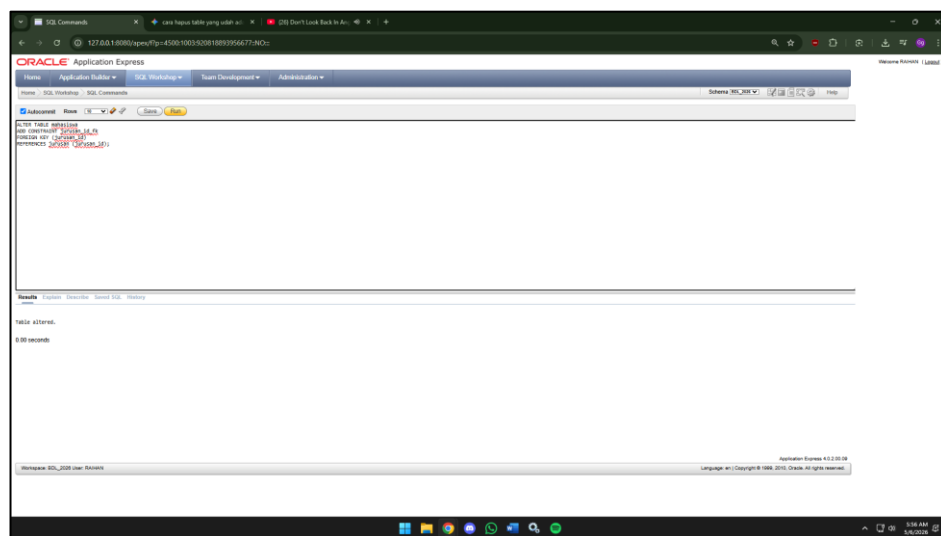
Query tersebut digunakan untuk membuat tabel jurusan dengan dua kolom, yaitu jurusan_id dan nama_jurusan. Melalui perintah CREATE TABLE, jurusan_id ditetapkan sebagai PRIMARY KEY, sedangkan nama_jurusan diberi aturan NOT NULL sehingga wajib diisi. Hasilnya, tabel jurusan berhasil dibuat dengan ketentuan kunci utama dan kolom yang harus terisi.

3. Menggunakan perintah `ALTER TABLE` untuk menambahkan `FOREIGN KEY CONSTRAINT` pada tabel mahasiswa.

Query:

```
ALTER TABLE mahasiswa  
ADD CONSTRAINT jurusan_id_fk  
FOREIGN KEY (jurusan_id)  
REFERENCES jurusan (jurusan_id);
```

Hasil:



Gambar 9.49 Tampilan penambahan `FOREIGN KEY jurusan_id`

Penjelasan:

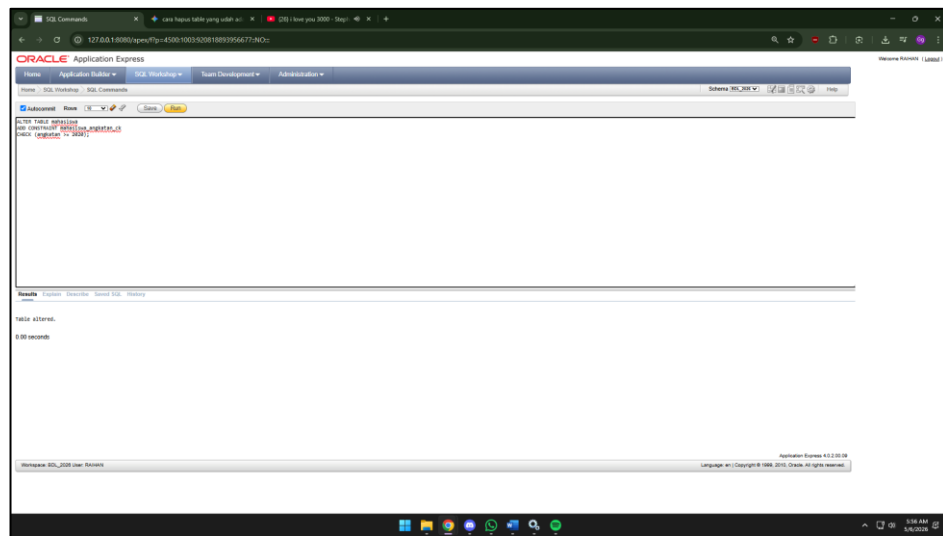
Query tersebut digunakan untuk menambahkan `CONSTRAINT` berupa `FOREIGN KEY` pada tabel mahasiswa. Melalui perintah `ALTER TABLE`, kolom `jurusan_id` pada tabel mahasiswa dihubungkan dengan kolom `jurusan_id` yang ada pada tabel jurusan. Dengan adanya aturan ini, setiap data `jurusan_id` pada tabel mahasiswa harus sesuai dengan data yang tersedia di tabel jurusan. Hasilnya, terbentuk hubungan antar tabel yang menjaga konsistensi dan integritas data.

4. Menggunakan perintah ALTER TABLE untuk menambahkan CHECK CONSTRAINT pada tabel mahasiswa.

Query:

```
ALTER TABLE mahasiswa  
ADD CONSTRAINT mahasiswa_angkatan_ck  
CHECK (angkatan >= 2020);
```

Hasil:



Gambar 9.50 Tampilan penambahan CHECK CONSTRAINT

Penjelasan:

Query tersebut digunakan untuk menambahkan CHECK CONSTRAINT pada tabel mahasiswa. Melalui perintah ALTER TABLE, ditetapkan bahwa nilai angkatan harus ≥ 2020 . Hasilnya, hanya data dengan angkatan sesuai aturan yang dapat disimpan.

9.5 Kesimpulan

Berdasarkan Berdasarkan praktikum yang telah dilakukan, dapat disimpulkan bahwa `CONSTRAINT` memiliki peran penting dalam menjaga keakuratan dan konsistensi data pada tabel. Jenis seperti `PRIMARY KEY`, `FOREIGN KEY`, `NOT NULL`, `UNIQUE`, dan `CHECK` digunakan untuk memastikan data sesuai aturan, misalnya `PRIMARY KEY` untuk identitas unik, `NOT NULL` agar kolom wajib diisi, `UNIQUE` mencegah duplikasi, `CHECK` membatasi nilai, dan `FOREIGN KEY` menghubungkan antar tabel.

Relasi antar tabel juga dapat dikelola dengan lebih baik menggunakan `ON DELETE CASCADE` dan `ON DELETE SET NULL`, sehingga konsistensi data tetap terjaga saat data induk dihapus. Selain itu, pengelolaan `CONSTRAINT` dapat dilakukan melalui perintah seperti `ENABLE`, `DISABLE`, `DROP CONSTRAINT`, hingga `CASCADE` sesuai kebutuhan.

Selain itu, `VIEW` berfungsi sebagai tabel virtual dari hasil *query*. Perintah seperti `CREATE VIEW` dan `CREATE OR REPLACE VIEW` digunakan untuk membuat atau memperbarui `VIEW`, sedangkan `WITH CHECK OPTION` dan `WITH not READ ONLY` mengatur batasan akses data. Teknik seperti *subquery*, *inline view*, `ROWNUM`, dan *TOP-N analysis* membantu menampilkan data secara lebih spesifik.